



DIPLOMATERVEZÉSI FELADAT

Mészáros Krisztina
Mérnökinformatikus hallgató részére

Ökoszisztéma szimulációja Unity3D alapon

A szimulációk lehetővé teszik a valós rendszerek viselkedésének biztonságos és költséghatékony vizsgálatát. Segítségükkel komplex folyamatokat modellezhetünk, különböző változókkal kísérletezhetünk, és megérthetjük a valós rendszerek működését anélkül, hogy a valóságban beavatkoznánk. Az ökoszisztéma-szimulációk különösen fontosak, mivel bemutatják a természetes egyensúly, a táplálkozási láncok és a populációdinamika összefüggéseit vizuálisan ábrázolható grafikonok formájában is. A Unity keretrendszer jó alapot biztosít egy ilyen szimuláció elkészítéséhez.

A hallgató feladata a korábban általa kidolgozott, Unity3D alapú ökoszisztéma szimulációs rendszer továbbfejlesztése, hogy az minél kiterjeszthetőbb legyen és faj-specifikus tulajdonságok és viselkedések legyenek benne. A részletes statisztikák vizuális megjelenítése szintén fontos szempont. Összefoglalva, a hallgató feladatának a következőkre kell kiterjednie:

- Röviden mutassa be a Unity keretrendszert és a korábban elkészült szimulációs megoldást!
- Fejlessze tovább a megoldást, hogy az minél kiterjeszthetőbb legyen! Ügyeljen a fontos események logolására is!
- Vezessen be faj-specifikus tulajdonságokat és viselkedéseket és vizsgálja meg, ezek hogyan befolyásolják a szimulációt!
- Készítsen részletes statisztikákat a szimulációk futása után! Gondoskodjon ezek vizuális megjelenítéséről is!

Tanszéki konzulens: Dr. Somogyi Ferenc Attila, adjunktus

Budapest, 2025. október 6.

Dr. Charaf Hassan
egyetemi tanár, tanszékvezető



Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Automatizálási és Alkalmazott Informatikai Tanszék

Ökoszisztéma szimulációja Unity3D alapokon

DIPLOMATERV

Készítette
Mészáros Krisztina

Konzulens
Dr. Somogyi Ferenc Attila

2026. június 1.

Tartalomjegyzék

Kivonat	i
Abstract	ii
1. Bevezetés	1
2. Specifikáció	3
2.1. A fejlesztési folyamat evolúciója	3
2.2. Funkcionális követelmények	4
2.3. Nem-funkcionális követelmények	4
2.4. A szimulációs paraméterek specifikációja	5
2.5. Felhasználói esetek (Use Cases)	5
2.6. Felhasznált technológiák	5
2.6.1. A Unity játékmotor és a komponens-alapú architektúra	5
2.6.2. C# szkriptelés és menedzselt futtatókörnyezet	6
2.6.3. InfluxDB idősr-adatbázis	7
2.6.4. Grafana analitikai és vizualizációs platform	8
3. Előzmények és irodalomkutatás	9
3.1. Irodalomkutatás és elméleti háttér	9
3.2. Hasonló alkotások és inspirációk	11
3.2.1. Sebastian Lague: Simulating Ecosystems	11
3.2.2. VAV: Evolúciós devlog	12
3.2.3. Fun Master Ed	12
3.2.4. Primer	12
3.3. Következtetések és tervezési irányelvek	13
4. A szimulációs rendszer architektúrája és technikai megvalósítása	15
4.1. Architekturális tervezés és osztályhierarchia	15
4.2. A ragadozó-préda dinamika kiterjesztése	17
4.3. Eseményvezérelt kommunikációs modell	18
4.4. A döntési folyamatok moduláris tervezése és implementációja	20
4.4.1. Véges Állapotú Gép (ClassicFSM) megvalósítása	21
4.4.2. Hasznossági alapú döntési rendszer (Utility AI)	21
4.4.3. Fuzzy Kognitív Térkép (FCM) tervezése	22
4.5. A populációgenetikai adatmodell	24
4.5.1. A Genome osztályok belső felépítése és kódolása	24
4.5.2. A mutációs algoritmus tervezése és matematikai modellje	25
4.5.3. Keresztezési (rekombinációs) mechanizmus	26
4.6. Elosztott Telemetria és Aszinkron Adatgyűjtés	26
4.6.1. A Unity főszál (Main Thread) védelme	26

4.6.2. A HTTP POST kérések felépítése és az alkalmazott InfluxDB Line Protocol	27
5. Az eredmények értékelése	29
5.1. Rendszer- és teljesítményelemzés (Kritikai reflexió)	29
5.2. A döntési modellek kísérleti összehasonlítása	29
5.3. Továbbfejlesztési lehetőségek	29
5.3.1. Technológiai irány: Skálázhatóság és a Unity DOTS architektúra . .	29
5.3.2. Biológiai és ökológiai irány: A szimulációs hűség növelése	30
5.3.3. Játékmenet és gamifikáció: Interaktív ökoszisztéma-szimulátor és Sandbox mód	30
5.4. Összefoglalás	31
Irodalomjegyzék	33

HALLGATÓI NYILATKOZAT

Alulírott *Mészáros Krisztina*, szigorló hallgató kijelentem, hogy ezt a diplomaterv meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem. A Függelékben egyértelműen megjelöltem, hogy a mesterséges intelligencia eszközeit alkalmaztam-e a dolgozat elkészítéséhez; amennyiben igen, annak módját és mértékét a táblázatban közöltem. Tudomásul veszem, hogy a mesterséges intelligenciával generált tartalomért – annak mértékétől függetlenül – teljes felelősséggel tartozom.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző(k), cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövege pedig a BME Címtárban regisztrált személyek számára elérhető legyen. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Budapest, 2026. június 1.

Mészáros Krisztina
hallgató

Kivonat

Diplomamunkám témája egy komplex, **Unity** alapú ökoszisztéma-szimulációs keretrendszer fejlesztése, amelynek elsődleges célja a populációgenetikai folyamatok és az öröklődő tulajdonságok vizsgálata digitális környezetben. A fejlesztés mostanra egy négyéves intervallumot ölel fel, amely során a szoftver egy egyszerű ágensalapú modellből egy robusztus, eseményvezérelt rendszerré fejlődött.

A projekt kezdeti szakaszában, vagyis az egyetemi BSc képzés keretein belül, egy kódgenerált környezetben élő préda-populáció (nyulak) viselkedésének modellezése történt meg. Az egyedek olyan öröklhető jellemzőkkel rendelkeztek, mint a mozgási sebesség, a látótávolság és különféle reprodukciós paraméterek, amelyek közvetlen hatást gyakoroltak a túlélési rátára és a természetes szelekcióra.

A egyetemi MSc képzés megkezdésével, a fejlesztés későbbi fázisába érve, a hangsúly előbb a szoftverarchitektúra optimalizálására és a refaktorálásra helyeződött. A rendszer kezdeti, monolitikus felépítését (úgynevezett „god class” probléma) egy moduláris, tiszta felelősségi körökkel rendelkező osztályszerkezet váltotta fel, jelentősen növelve a kód karbantarthatóságát és bővíthetőségét.

A biológiai hűség fokozása érdekében a szimuláció egy ragadozó fajjal (rókák) egészült ki, megteremtve a predator–prey dinamikát. A kialakuló populáció-oszcillációk vizsgálata során a Lotka–Volterra modell elméleti alapjait alkalmaztam a rendszer validálására. A szoftver legfrissebb változata már eseményvezérelt architektúrára épül, amely nemcsak a számítási hatékonyságot javítja az állapotellenőrzések minimalizálásával, hanem rugalmas keretet biztosít az új viselkedésformák implementálásához is.

A szimuláció futása során keletkező adatokat a rendszer **InfluxDB** idősoros adatbázisban tárolja, az adatok vizualizációja és a valós idejű statisztikai elemzés pedig **Grafana** segítségével valósul meg. Ez a megoldás lehetővé teszi a komplex ökológiai folyamatok transzparens nyomon követését és a különböző döntéshozatali modellek összehasonlítását.

Abstract

The subject of my thesis is the development of a complex, **Unity**-based ecosystem simulation framework, primarily aimed at investigating population genetic processes and heritable traits within a digital environment. The development now spans a four-year period, during which the software has evolved from a simple agent-based model into a robust, event-driven system.

In the initial phase of the project, conducted within the framework of my BSc studies, the behavior of a prey population (rabbits) living in a code-generated environment was modeled. Individuals possessed heritable characteristics such as movement speed, vision range, and various reproductive parameters, which had a direct impact on survival rates and natural selection.

Upon commencing my MSc studies and entering a later phase of development, the focus shifted toward software architecture optimization and refactoring. The system's initial monolithic structure (the so-called "god class" problem) was replaced by a modular class hierarchy with clean responsibilities, significantly increasing code maintainability and extensibility.

To enhance biological fidelity, the simulation was expanded to include a predator species (foxes), establishing predator-prey dynamics. During the investigation of the resulting population oscillations, I applied the theoretical foundations of the Lotka-Volterra model to validate the system. The latest version of the software is built on an event-driven architecture, which not only improves computational efficiency by minimizing state checks but also provides a flexible framework for implementing new behavioral forms.

Data generated during the simulation is stored in an **InfluxDB** time-series database, while data visualization and real-time statistical analysis are realized using **Grafana**. This solution enables the transparent monitoring of complex ecological processes and the comparison of different decision-making models.

1. fejezet

Bevezetés

A modern tudomány egyik legnagyobb kihívása a komplex biológiai rendszerek dinamikájának megértése és előrejelzése. Az ökoszisztémák vizsgálata a valóságban gyakran akadályokba ütközik: a folyamatok időigényesek, a beavatkozások pedig visszafordíthatatlanok vagy etikailag megkérdőjelezhetők lehetnek. A számítógépes szimulációk ezzel szemben biztonságos, költséghatékony és reprodukálható környezetet biztosítanak a populációgenetikai és ökológiai elméletek teszteléséhez.

A diplomamunka alapvető motivációja egy olyan digitális laboratórium létrehozása volt, amely képes vizuálisan és adat szintjén is reprezentálni a természetes szelekciót és a fajok közötti interakciókat. A projekt célja nem csupán egy látványos vizualizáció elkészítése, hanem egy olyan robusztus szoftverarchitektúra kialakítása, amely alkalmas lehet tudományos igényű adatgyűjtésre és elemzésre.

A projektem gyökerei a BSc képzés idejére nyúlnak vissza, ahol az első ágensalapú modellek implementálása megtörtént. A koncepció kidolgozásakor inspirációként szolgált Sebastian Lague ismert ökoszisztéma-szimulációs projektje, amely rávilágított a Unity3D motorban rejlő lehetőségekre a biológiai modellezés terén.

Míg a kezdeti inspiráció a vizuális reprezentációra és az alapvető túlélési logikára fókuszált, a saját fejlesztésem hamar túllépett ezen a kereten. Az eddigi megoldásokhoz képest a hangsúly eltolódott a szoftvertechnológiai precizitás, a skálázhatóság és a professzionális adatkezelés irányába. A korábbi monolitikus struktúrát felváltotta egy moduláris felépítés, amely lehetővé tette a komplexebb, fajspecifikus viselkedésformák és az eseményvezérelt működés integrálását.

A dolgozatban bemutatott rendszer egy Unity3D alapú, eseményvezérelt ökoszisztéma-szimuláció. A keretrendszer legfőbb újdonsága a populációgenetikai folyamatok mély integrációja: az egyedek DNS-alapú öröklődési mechanizmusokkal rendelkeznek, amelyek meghatározzák fizikai jellemzőiket és viselkedési stratégiáikat.

A szimuláció során keletkező hatalmas mennyiségű adat hatékony kezelésére és valós idejű monitorozására egy InfluxDB adatbázisra épülő naplózó rendszert fejlesztettem ki. Az események továbbítása HTTP-alapú API hívásokon keresztül történik, az adatok vizuális megjelenítését és elemzését pedig egy Grafana dashboard biztosítja. Ez a felépítés lehetővé teszi, hogy a populációk változásait, a mutációk hatásait és a ragadozó-préda dinamikát ne csak a Unity környezetben, hanem külső statisztikai eszközökkel validált, interaktív grafikonok formájában is elemezhesük.

A dolgozat szerkezete a következőképpen alakul: A **2. fejezet** a feladatkiírás részletes elemzésével és a követelmények specifikációjával foglalkozik, majd a **3. fejezet** mutatja be az elméleti hátteret, a hasonló megoldásokat (különös tekintettel Sebastian Lague munkásságára), valamint az olyan biológiai alapmodelleket, mint a Lotka–Volterra egyenletek. A **?? fejezet** tartalmazza a rendszertervet, a szoftverarchitektúra döntési pontjait és az ese-

ményvezérelt működés logikáját. A megvalósítás technikai lépéseit a Unity környezetben, kitérve a faj-specifikus viselkedések programozására, a **?? fejezet** részletezi. Végezetül a **?? fejezet** a szimuláció futtatása során nyert adatok statisztikai elemzését, a rendszer validálását és a továbbfejlesztési lehetőségek bemutatását tartalmazza.

2. fejezet

Specifikáció

Ebben a fejezetben a diplomaterv kitűzött céljait bontom le konkrét követelményekre. Mivel jelen dolgozat egy négy féléven átívelő kutatási és fejlesztési folyamat (*Önálló Laboratórium 1-2*, majd *Diplomatervezés 1-2*) szintézise, a követelmények elemzése során éles határvonalat kell húzni a kiindulópontként szolgáló alaprendszer, valamint a teljes MSc képzés során elért fokozatos mérnöki előrelépések között.

2.1. A fejlesztési folyamat evolúciója

A szoftverrendszer nem egyetlen fázisban nyerte el végső formáját. A specifikáció pontos megértéséhez így elengedhetetlen a fejlesztési mérföldkövek áttekintése, meghúzva az éles határvonalat a kiindulópontot jelentő szakdolgozat és a mesterképzés során végzett kutató-fejlesztő munka között:

Kiindulási állapot (BSc fázis): Egy egyszerűsített, monolitikus architektúrájú szimuláció, amely kizárólag egyetlen préda faj (nyulak) jelenlétét kezelte egy kódgenerált szigeten, merev, determinisztikus (FSM-alapú) döntési logikával és lokális, strukturálatlan adatmentéssel.

Továbbfejlesztés és kutatás (MSc fázis): A mesterképzés során a rendszer teljes körű strukturális és funkcionális metamorfózison ment keresztül. A fejlesztési eredmények az alábbi főbb pillérekre oszthatók:

- *Architekturális refaktorálás:* Megtörtént a „god class” minták felszámolása és a felölősségi körök szétválasztása (Single Responsibility Principle). Az erőforrás-kezelési problémák elhárítására, valamint a komponensek közötti laza csatolás biztosítására a rendszer eseményvezérelt kommunikációs architektúrát kapott.
- *Ökoszisztéma-modell bővítése:* Bevezetésre került a ragadozó faj (rókák), amely maga után vonta az új érzékelési, menekülési és vadászati algoritmusok implementálását. Az ágensek biológiai modellje életerő-alapú (health-based) rendszerrel, valamint új, generációkon át öröklődő fenotípusos tulajdonságokkal (pl. álcázás, bátorság...) egészült ki.
- *Döntési architektúrák absztrakciója:* A merev FSM-en túlmutatóan kiépítésre került a hasznossági alapú (Utility-based AI), valamint a Fuzzy Kognitív Térkép (FCM) elvén működő döntési mechanizmus. A három különböző gondolkodásmód stabil, a szimuláció indításakor szabadon konfigurálható, párhuzamosan futtatható és egymással versenyztethető ágens-architektúrává vált.

- *Elosztott telemetria és adatgyűjtés:* A kezdeti lokális, fájlalapú (CSV) naplózást egy modern, elosztott telemetria-rendszer váltotta fel. A szimulációs adatok valós időben, aszinkron HTTP kliensen keresztül továbbítódnak egy külső idősoros adatbázisba (InfluxDB), az adatok tudományos igényű vizualizációját és elemzését pedig egy dedikált Grafana dashboard látja el.

2.2. Funkcionális követelmények

A fenti evolúciós folyamat eredményeképpen a lezárt mérnöki rendszernek a következő funkcionális követelményeket kell teljesítenie:

- **Többszintű populációdinamika és interakciók:** Legalább két egymásra utalt faj (préda és ragadozó) egyidejű szimulációja folyamatos térbeli interakciókkal (táplálkozás, vadászat, szaporodás, elhullás...).
- **Dinamikus cserélhető döntési architektúra:** Az ágensek számára biztosítani kell a lehetőséget, hogy három különböző logikai struktúra (Véges állapotgép, Hasznossági modell, vagy Fuzzy Kognitív Térkép) szerint hozzák meg döntéseiket, lehetővé téve ezen mechanizmusok közvetlen összehasonlítását.
- **Genetikai öröklődés és mutáció:** Az egyedek fizikai tulajdonságai (sebesség, látótávolság) és döntési paraméterei (súlyok, gráf-élek) genotípus alapú (**Genome**) tárolása, kombinálása és mutációval kísért átöröklése az utódokba.
- **Aszinkron külső adatkapcsolat:** A szimuláció futása közben keletkező események (születés, elhullási okok, populációméret, genetikai drift...) strukturált, valós idejű továbbítása külső tároló felé, a szimulációs főszál blokkolása nélkül.

2.3. Nem-funkcionális követelmények

A szoftverarchitektúra minőségével szemben támasztott elvárások:

- **Kiterjeszthetőség (Extensibility):** A rendszernek meg kell felelnie az Open/Closed elvnek. Új fajok, tulajdonságok vagy alternatív döntési mechanizmusok bevezetése nem igényelheti a meglévő szimulációs mag módosítását.
- **Teljesítmény és Skálázhatóság:** A szimulációnak stabil képkockasebesség (FPS) mellett kell futnia több száz egyidejűleg aktív ágens jelenléte esetén is. Emiatt a hagyományos, **Update**-alapú környezetlekérdezést (polling) eseményvezérelt (C# delegátumokra és eventekre épülő) architektúrával kell megvalósítani.
- **Valós idejűség és Leválasztott I/O:** A statisztikai adatok gyűjtése, formázása és a hálózati HTTP kommunikáció menedzselése nem okozhat mikroszagatásokat (stuttering) vagy késleltetést a Unity vizuális megjelenítési folyamataiban.
- **Karbantarthatóság és Alacsony Csatolás (Low Coupling):** Az egyedek vizuális megjelenítésének, fizikai reprezentációjának és belső döntési logikájának szigorúan elválasztott komponensekként kell működniük, elkerülve a korai fejlesztési szakaszokra jellemző monolitikus struktúrákat.

2.4. A szimulációs paraméterek specifikációja

A pontosabb elemzés érdekében rögzíteni kell azokat a változókat, amelyeket a rendszernek kezelnie kell. A szimuláció bemeneti adatait és a vizsgált változók rendszerezett listáját a 2.1. táblázat foglalja össze.

Kategória	Paraméterek
Környezet	Terület mérete, táplálék (fű) újratermelődési rátája
Egyed (Genetika)	Mozgási sebesség, látómező szöge/távolsága, éhségküszöb
Új Genetikai Jegyek	Lopakodás (<i>stealth</i>), álcázás (<i>camouflage</i>), bátorság (<i>bravery</i>)
Populáció	Kezdeti egyedszám, mutációs ráta, választott gondolkodásmód
Rendszer	Logolási gyakoriság, szimulációs időlépték (<i>Timescale</i>), InfluxDB kliens beállítás

2.1. táblázat. A szimuláció bővített kulcsparaméterei

A táblázatban felsorolt paraméterek finomhangolása alapvetően meghatározza a szimulált ökoszisztéma stabilitását. A jelenlegi fázis célja éppen ezen paraméterek hosszú távú hatásainak vizsgálata a különböző döntési architektúrák tükrében.

2.5. Felhasználói esetek (Use Cases)

A rendszer elsődleges felhasználója a kutató/hallgató, aki a következő műveleteket végzi:

- Konfiguráció:** A szimuláció indítása előtt megválasztja a környezeti paramétereket, a kezdeti egyedszámokat és az ágensek által használt specifikus döntési modellt (FSM, Utility, FCM).
Megfigyelés: Valós időben, 3D-s környezetben követi a vizuális térben zajló populációdinamikai folyamatokat és egyedi interakciókat.
- Valós idejű elemzés:** A szimuláció futásával párhuzamosan, a külső Grafana dashboardon megjelenő valós idejű idősoros grafikonok és statisztikák (pl. átlagéletkor, elhullási okok aránya, genetikai értékek eloszlása) alapján kiértékeli a választott döntési rendszer stabilitását.

2.6. Felhasznált technológiák

A szimulációs keretrendszer megvalósítása komplex szoftverarchitekturális döntéseket igényelt. Mivel a projekt célja nemcsak egy elméleti populációdinamikai modell felállítása, hanem egy vizuálisan is követhető, valós idejű telemetriával ellátott 3D környezet létrehozása volt, a fejlesztés során az iparágban elterjedt, nagy teljesítményű eszközök integrációjára volt szükség.

2.6.1. A Unity játékmotor és a komponens-alapú architektúra

A szimuláció magját a *Unity* játékmotor képezi. A Unity választását elsősorban a fejlett 3D-s renderelő motorja, a beépített fizikai alrendszer, valamint az objektumorientált, komponens-alapú szoftverfejlesztési paradigmája indokolta.

A hagyományos objektumorientált programozás merev osztályhierarchiáival szemben a Unity a **Composition over Inheritance** elvét követi a **színterek** (*Scenes*) és **játékobjektumok** (*GameObjects*) szintjén.

- **GameObject:** A Unity szintén legkisebb atomi egysége, amely önmagában nem hordoz semmilyen specifikus viselkedést vagy vizuális reprezentációt. Egy üres konténerként funkcionál, amelynek egyetlen kötelező tulajdonsága a térbeli pozíciót, forgatást és skálázást leíró **Transform** komponens.
- **Component:** Az objektumok funkcionalitását a hozzájuk csatolt komponensek határozzák meg. Egy ágens esetében külön komponens felel a vizuális megjelenítésért (**MeshRenderer**), a térbeli ütközések detektálásáért (**Collider**), valamint például a mesterséges intelligencia döntési logikájáért. Ez a moduláris felépítés rendkívül rugalmas fejlesztést tesz lehetővé, hiszen az ágensek tulajdonságai dinamikusan, futásidőben is változtathatók (pl. új komponens hozzáadásával vagy eltávolításával).

MonoBehaviour életciklus és működési mechanizmus

A Unity-ben írt egyedi szkriptek mindegyike a natív **MonoBehaviour** alapsztályból öröklődik. Ez az osztály biztosítja a hidat a C# kód és a Unity belső, C++ alapú motorja között. A **MonoBehaviour** alapú komponensek egy szigorúan meghatározott, eseményvezérelt életciklus-függvénymátrix alapján futnak, amelyet a Unity fő végrehajtási szála (*Main Thread*) menedzsel.

A szimuláció szempontjából a legfontosabb életciklus-fázisok a következők:

- **Awake()** és **Start():** Az objektum inicializálásakor, pontosan egyszer lefutó függvények. A rendszer itt építi fel a belső referenciákat (például az ágens itt kéri le a saját **Sensor** vagy **Eat** komponenseinek referenciáit).
- **Update():** Minden egyes renderelési képkockánál (*frame*) egyszer meghívódó ciklus. Itt zajlanak a nem-fizikai kalkulációk, mint például az ágensek belső állapotának mintavételezése vagy a bemeneti perifériák kezelése. Futási gyakorisága hardverfüggő (változó FPS).
- **FixedUpdate():** A **Update()**-tal ellentétben ez a ciklus determinisztikus, fix időközönként hívódik meg (alapértelmezetten 0.02 másodpercenként), teljesen függetlenül a renderelési képkockasebességtől. A szimuláció fizikai interakciói – mint a sugárkövetéses látómező-ellenőrzés (**Physics.Raycast**) és a mozgásvektorok kiszámítása – ebben a ciklusban futnak a konzisztens fizikai viselkedés biztosítása érdekében.

2.6.2. C# szkriptelés és menedzselt futtatókörnyezet

A Unity elsődleges programozási nyelve a C#, amely a modern, típusbiztos és nagy teljesítményű szoftverfejlesztés alapköve. A Unity a *Mono* vagy az *IL2CPP* (*Intermediate Language to C++*) keresztfordító technológiát használja a kód futtatására.

A szimulációs szoftver tervezése során a C# nyelv számos fejlett funkcióját ki kellett aknázni:

- **Eseményvezérelt architektúra (*Events, Actions*):** Az ágensek közötti és a belső rendszerek közötti kommunikáció leválasztott módon, C# *event*-eken keresztül valósul meg.

Például, amikor egy ágens éhsége eléri a kritikus szintet, az **Eat** komponens kivált egy **OnHungerCritical** eseményt, amelyre az **AnimalEventHandler** iratkozik fel, és módosítja az ágens globális állapotát. Ez megakadályozza a komponensek közötti káros szoros kötődést.

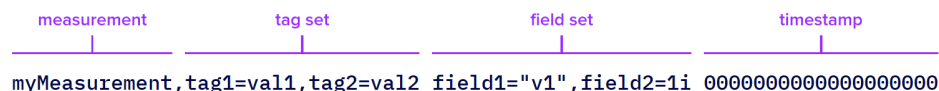
- **Polimorfizmus és absztrakció:** Az ökoszisztéma különböző fajai (pl. Bunny, Fox) egy közös absztrakt `Animal` alaposztályból származnak, megosztva az alapvető életfenntartó logikákat, de egyedileg felüldefiniálva a specifikus tulajdonságokat, mint a préda-ragadozó maszkok vagy a fajspecifikus táplálkozási rétegek.
- **Memóriakezelés és a Garbage Collector terhelése:** Mivel a szimuláció során folyamatosan ágensek születnek és halnak meg, a memóiafoglalások optimalizálása kritikus. A C# menedzselt környezetéből adódó automatikus szemétgyűjtés futás közbeni mikroakadásokat (*GC spike*) okozhat. Ennek elkerülésére a rendszer minimalizálja a futásidőben történő dinamikus memóiafoglalásokat (*heap allocation*), előnyben részesítve a struktúrák használatát és a fix tömbméreteket.

2.6.3. InfluxDB idősör-adatbázis

Az InfluxDB egy nyílt forráskódú, kifejezetten idősör-adatok (*Time Series Data*) tárolására és elemzésére optimalizált NoSQL adatbázis-rendszer. A hagyományos relációs adatbázisokkal vagy a strukturálatlan CSV fájlokkal szemben az InfluxDB belső felépítése (*Time-Structured Merge Tree*) úgy lett kialakítva, hogy másodpercenként akár több százezer írási műveletet is képes legyen feldolgozni minimális erőforrás-igénnyel.

Az adatbázis az adatokat egy REST API-n keresztül, HTTP POST kérések segítségével, úgynevezett *Line Protocol* formátumban fogadja. Ez egy szigorúan meghatározott, szöveges alapú formátum, amely minimalizálja a hálózati overheadet. Egy rögzített adatpont elméleti szerkezete a következő elemekből áll:

- **Measurement (Mérés):** A relációs adatbázisok táblaneveivel analóg strukturális elem, amely meghatározza a rögzíteni kívánt esemény vagy metrika alapvető kontextusát.
- **Tag-ek (Címkék):** Kulcs-érték párok formájában megadott metaadatok, amelyeket az InfluxDB **automatikusan indexel**. Mivel az indexelés miatt ezeken a mezőkön a lekérdezések futási ideje konstans vagy logaritmikus, ide a diszkrét, kategorizálható tulajdonságokat érdemes helyezni. A tag-ek értékei mindig sztring típusúak.
- **Field-ek (Mezők):** Kulcs-érték párok, amelyek a **ténylegesen mért, folyamatosan változó célértékeket** hordozzák. A mezők értékei nem indexeltek, de rajtuk matematikai és statisztikai aggregációk (átlagolás, minimum- és maximumkeresés) végezhetők. Értékük lehet lebegőpontos szám, egész szám, logikai érték vagy sztring.
- **Timestamp (Időbélyeg):** Az esemény bekövetkezésének pontos ideje UNIX időbélyeg formátumban.



```

measurement      tag set      field set      timestamp
|               |           |           |
myMeasurement,tag1=val1,tag2=val2 field1="v1",field2=1i 00000000000000000000

```

2.1. ábra. Az InfluxDB Line Protocol felépítése

A szintaxis szempontjából kritikus pont a **szóköz karakterek** elhelyezkedése: az első szóköz választja el az indexelt metaadatokat (Measurement és Tag-ek) a nem indexelt értékektől (Field-ek), míg a második szóköz különíti el a mezőket a záró időbélyegtől.

A Tag-ek és Field-ek listáján belüli elemeket vesszővel (,), a kulcs-érték párokat pedig egyenlőségjellel (=) kell elválasztani, szóközők nélkül.

A Unity és az InfluxDB integrációját egy aszinkron, korutin-alapú (`IEnumerator`) hálózati kliens valósítja meg, amely a `UnityWebRequest` könyvtár segítségével, a háttérben küldi el az összegyűjtött adatokat, így az hálózati I/O műveletek nem blokkolják a Unity fő futási szálát (*Main Thread*). A konkrét implementációt majd a ?? fejezet mutatja be részletesebben.

Az így kiküldött adatok közvetlenül az InfluxDB `bucket`-jébe¹ kerülnek, ahonnan a Grafana dashboardok valós időben, másodperces felbontással képesek kirajzolni a populáció túlélési és adaptációs grafikonjait.

2.6.4. Grafana analitikai és vizualizációs platform

Míg az InfluxDB a nagy sebességű adattárolásért felel, addig a gyűjtött adatok valós idejű monitorozását, aggregációját és vizuális elemzését a *Grafana* biztosítja. A Grafana egy nyílt forráskódú, multiplatform analitikai és dashboard-tervező szoftver, amely natív módon, optimalizált lekérdezéseken keresztül képes kapcsolódni az InfluxDB adatforráshoz.

A szimulációs keretrendszerben a Grafana alkalmazása több szempontból is kritikus mérnöki döntés volt:

- **Valós idejű megfigyelés (*Real-time Dashboards*):** A kutató vagy felhasználó anélkül követheti nyomon a populációk állapotát, az átlagos éhségszinteket vagy a mutációs ráták alakulását, hogy meg kellene szakítania a szimuláció futását. A Grafana automatikusan, előre definiált időközönként frissíti a grafikonokat.
- **Komplex adatelemzés Flux lekérdezőnyelvvvel:** A Grafana felületén az InfluxDB *Flux* funkcionális adatlekérdező nyelve segítségével bonyolult aggregációk és matematikai statisztikák is kiszámíthatók futásidőben (például mozgóátlagok számítása az ágensek sebességére, vagy a halálokok százalékos megoszlása egy adott időablakban).
- **Rendszerfüggetlen kiértékelés:** Mivel a Grafana egy böngészőből elérhető webes felületet biztosít, a szimuláció vizuális analízise teljesen leválasztható a Unity-t futtató hardver erőforrásairól. Ez lehetővé teszi, hogy a szimuláció akár egy headless szerveren vagy külön szálon fusson, míg a kiértékelés egy külső monitoron vagy eszközön történik.

¹Az InfluxDB kontextusában a *bucket* egy logikai adattároló egység (analóg a relációs adatbázisok adatbázis-fogalmával), amely meghatározott adatmegőrzési szabályokkal (*Retention Policy*) rendelkezik, és idősoros adatok (*time series data*) strukturált tárolására szolgál.

3. fejezet

Előzmények és irodalomkutatás

A szimulációs rendszerek tervezése előtt alapos irodalomkutatást végeztem az ACM Digital Library adatbázisában ¹, valamint megvizsgáltam a területen elérhető népszerű szoftveres megoldásokat és oktatási célú projekteket. A kutatás célja az volt, hogy feltérképezzem az ágensalapú modellezés (Agent-Based Modeling – ABM) aktuális módszereit és a döntéshozatali algoritmusok implementációs lehetőségeit.

3.1. Irodalomkutatás és elméleti háttér

Az ökoszisztémák modellezése és az ágensalapú szimulációk területén végzett kutatásaim során több olyan tudományos publikációt vizsgáltam meg, amelyek alapjaiban határozták meg a fejlesztés irányvonalát. Az alábbiakban ezeket a munkákat részletezem, kiemelve a saját megoldásomtól való eltéréseket.

Tiago és Reis munkájukban [4] a koevolúció folyamatát vizsgálták, ahol a ragadozó és préda fajok egyidejűleg fejlődnek genetikai programozás és döntési fák segítségével. A cikk fő fókusza a stratégiai viselkedésformák kialakulása volt: a kutatók azt elemezték, hogy a fajok hogyan képesek komplex válaszokat adni egymás változó taktikáira. Bár a genetikai megközelítés náluk is központi szerepet játszik, az én megoldásom lényegesen eltér a viselkedés leírásának módjában. Míg ők döntési fákat generáltak evolúciós úton, én egy rögzített, de paramétereiben (genotípus) örökölhető és mutálható szabályrendszert alkottam meg. Ez lehetővé teszi a populációgenetikai adatok (példából: látótávolság, sebesség) sokkal finomabb, folytonos skálán történő követését és statisztikai elemzését, szemben a diszkrét döntési ágakkal.

Bearss és társai [1] a klasszikus, Wilensky-féle „Wolf-Sheep Predation” modellt ültették át modern környezetbe egy oktatási gyakorlat keretében. Tanulmányuk rávilágított arra, hogy az ágensalapú modellezés (ABM) hatékonysága nagyban függ a választott keretrendszer számítási kapacitásától és vizualizációs képességeitől. Az ő munkájuk elsősorban a modell validálására és az oktatási módszertanra fókuszált. Ezzel szemben az én fejlesztésem nem csupán egy meglévő modell újrainplementálása, hanem egy kiterjeszthető mérnöki keretrendszer létrehozása. Míg az említett munka megmarad a hagyományos 2D-s rácsalapú szemléletnél, az én szimulációm a Unity3D motor lehetőségeit kihasználva folytonos 3D-s térben, procedurálisan generált domborzaton zajlik, ami merőben más útvonalkeresési és interakciós logikát igényel.

A projekt szempontjából a legmeghatározóbb Gras és szerzőtársai publikációja [5] volt, amely a Fuzzy Cognitive Map (FCM) alkalmazását mutatja be egyén alapú

¹Az ACM (Association for Computing Machinery) Digital Library a világ egyik legjelentősebb informatikai tudományos adatbázisa, amely hozzáférést biztosít neves szakfolyóiratokhoz és konferenciakiadványokhoz: <https://dl.acm.org/>

ökoszisztéma-szimulációkban. A cikk központi gondolata, hogy a biológiai lények döntéshozatala nem bináris logikát követ, hanem belső állapotok (pl. félelem, éhség, fáradtság) és külső ingerek súlyozott eredője. Bár a viselkedésmodellezés alapelveit tőlük kölcsönöztem, a megvalósítás technikai háttere és a vizsgált módszerek köre jelentősen különbözik. Grasék kutatása egy dedikált, tudományos célú szoftverkörnyezetben készült, ahol a vizualizáció és a szoftverarchitektúra rugalmassága másodlagos volt. Ezzel szemben az én munkámban a hangsúly a szoftvertechnológiai modernizáción és a különböző döntési paradigmák összehasonlíthatóságán van.

A fejlesztés során egy olyan moduláris keretrendszert alakítottam ki, amely lehetővé teszi három különböző döntési mechanizmus párhuzamos vizsgálatát és összehasonlítását:

- **Véges állapotgép (FSM):** A BSc szakaszban kidolgozott megoldás továbbvitele, ahol az ágensek viselkedését előre rögzített állapotátmeneti szabályok határozzák meg. Bár a módszer stabil, mérnöki szempontból viszont rugalmatlan, mivel nem teszi lehetővé az egyedi adaptációt és az evolúciós fejlődést.
- **Hasznossági alapú döntéshozatal (Utility AI):** Ebben a modellben az ágensek különböző súlyozott szempontok alapján számítják ki az egyes cselekvések várható hasznosságát. A súlyokat egy különálló, genotípust leíró osztály (**Genome**) tárolja, amely az öröklődés során mutálódni képes. Ez a megközelítés már lehetőséget ad az ökoszisztéma evolúciós fejlődésének megfigyelésére.
- **Fuzzy Cognitive Map (FCM):** A legkomplexebb alkalmazott módszer, amelyben a döntéshozatalt egy irányított, súlyozott gráf reprezentálja. Itt a belső állapotok és fogalmak egymásra gyakorolt hatása révén apró változások is finom, nem-lineáris viselkedésformákat eredményezhetnek. A rendszer tervezésekor külön figyelmet fordítottam ezen komplex gráfszerkezet hatékony tárolására és az eseményvezérelt architektúrába való integrálására.

A saját megoldásom újdonsága, hogy ezeket a mechanizmusokat egy egységes, eseményvezérelt Unity-architektúrába integráltam, kiegészítve real-time HTTP alapú naplózással. Ezáltal lehetőség nyílik arra, hogy a statisztikai elemzés során számszerűsíthető legyen az egyes "gondolkodásmódok" hatása a populáció túlélési rátájára és stabilitására.

A három architektúra közötti alapvető szoftvertechnológiai és elméleti különbségeket a 3.1. táblázat foglalja össze részletesen:

3.1. táblázat. Az FSM, Utility AI és FCM döntési architektúrák összehasonlítása

	Véges állapotgép (FSM)	Hasznossági AI (Utility)	Fuzzy Cognitive Map (FCM)
Alapvető működési elv	Diszkrét állapotok és determinisztikus átmenetek összessége.	Súlyozott matematikai függvények és hasznossági értékek maximum-keresése.	Koncepciók (belső állapotok és akciók) irányított, súlyozott gráfja.
Döntéshozatali logika	Feltételvezérelt (if-then), merev küszöbértékek alapján.	Folytonos, normalizált értékek lineáris kombinációja.	Iteratív, nem-lineáris állapotfrissítés szigmoid aktivációs függvényvel.
Viselkedési determinizmus	Teljesen determinisztikus és statikus; azonos ingerre mindig azonos válasz.	Dinamikus: az egyedi genom súlyai miatt azonos állapot más döntést szülhet.	Emergens és kontextusfüggő: a gráfon belüli visszacsatolások egyedi mintákat adnak.

Folytatás a következő oldalon...

3.1. táblázat. Az FSM, Utility AI és FCM döntési architektúrák összehasonlítása

	Véges állapotgép (FSM)	Hasznossági AI (Utility)	Fuzzy Cognitive Map (FCM)
Paraméterek reprezentációja	<i>Hardcoded</i> logikai szabályok és konstansok.	Elkülönített, öröklhető és mutálható leíró osztály: <i>UtilityGenome</i> .	Súlymatrixtól és topológiától függő egyedi tömb: <i>FCMGenome</i> .
Evolúciós alkalmaság	Nem alkalmas; a struktúra futásidőben nem alakítható vagy finomhangolható.	Kiválóan alkalmas; a súlyok folytonos skálán való mutációja stabilan követhető.	Összetett; a súlyok mutációja mellett a topológia stabilitása nehezen garantálható.
Szoftver-architektúrális előnyök	Rendkívül alacsony számítási igény, egyszerű és gyors implementáció.	Moduláris, könnyen skálázható, az ágensek szintjén jól elkülöníthető.	Komplex, nem-lineáris visszacsatolások és finom viselkedésformák modellezése.
Mérnöki korlátok és hátrányok	Merev, skálázhatatlan; komplex rendszernél „állapotrobbanás” lép fel.	Nem veszi figyelembe az egyes döntési tényezők egymásra gyakorolt hatásait.	Magasabb számítási és memóriagény, összetett debugolás és finomhangolás.

3.2. Hasonló alkotások és inspirációk

A szoftverfejlesztői közösségben, különösen a Unity3D motor köré csoportosulva, több figyelemre méltó projekt is foglalkozik ökológiai szimulációval. Ezek az alkotások jelentős inspirációt nyújtottak a fejlesztés során, ugyanakkor rávilágítottak olyan technikai korlátokra is, amelyeket jelen munka igyekszik feloldani.

3.2.1. Sebastian Lague: Simulating Ecosystems

A projekt elsődleges kiindulópontját Sebastian Lague munkássága [6] jelentette. Az általa bemutatott szimuláció sikeresen demonstrálta az ágensalapú modellezés alapvető törvényszerűségeit egy letisztult vizuális környezetben.

Bár jelen dolgozat alapgondolatai sok ponton hasonlítanak Lague megoldásához, szoftvertechnológiai és környezeti szempontból jelentős eltérések mutatkoznak. Míg Lague implementációja elsősorban a koncepció közérthető bemutatására és az egyszerűsége fókuszált, jelen rendszer egy robusztusabb, ipari sztenderdekhez közelebb álló mérnöki megközelítést alkalmaz.

Környezeti szempontból Lague munkája egy sík, határolt 3D-s térben zajlik, ezzel szemben az én rendszeremben az ágensek egy komplex, procedurálisan generált 3D-s domborzaton mozognak. Ez jelentősen megnöveli a navigációs algoritmusok (NavMesh) és a térbeli interakciók számítási igényét.

Szoftverarchitektúrális szempontból a legfontosabb különbség az ágensek közötti kommunikáció és a környezetérzékelés megvalósítása. Lague szimulációja erősen támaszkodik a folytonos lekérdezésre (polling): az ágensek a `Unity Update()` és `FixedUpdate()` ciklusai-ban, folyamatosan futó távolságmérésekkel és környezeti lekérdezésekkel határozzák meg a környezetük állapotát, ami magas ágensszám esetén jelentős teljesítménycsökkenéshez vezet. Jelen fejlesztés során ezt egy eseményvezérelt működéssel (C# eventek és delegátumok használatával) váltottam ki, ahol az ágensek állapota és döntési logikája csak releváns események (például táplálék generálódása vagy egy másik ágens pusztulása) hatására frissül. Ez a megközelítés drasztikusan javítja az erőforrás-kezelést és a skálázhatóságot.

Emellett a szimuláció megfigyelhetősége is szintet lépett: a statisztikai adatok gyűjtése nem egyszerű lokális fájlba írással történik, hanem egy professzionális idősoros adatbázisba (InfluxDB) továbbítódik HTTP protokollon keresztül. Ez lehetővé tette a Grafana alapú

dashboardok használatát, ahol a populációdinamikai mutatók valós időben, interaktív módon elemezhetők, messze túlmutatva a hivatkozott munka vizualizációs lehetőségein.

3.2.2. VAV: Evolúciós devlog

VAV fejlesztése [8] Lague alapjaira építve mutat be egy továbbfejlesztett modellt. Ebben a megoldásban kiemelkedő a morfológiai jellemzők — például az állatok nyakhosszának — öröklődése és környezethez való adaptációja.

Jelen munka szempontjából ezen projekt tanulsága az volt, hogy a vizuális jegyek és a funkcionális tulajdonságok közötti kapcsolat (például a testfelépítés és az elért táplálék viszonya) jelentősen növeli a szimuláció biológiai hűségét. Noha a jelenlegi fázisban a hangsúly inkább a populációgenetikai adatok statisztikai elemzésén és a szoftverarchitektúrán van, a VAV által bemutatott morfológiai diverzitás a későbbi továbbfejlesztési irányok egyik alapkövét jelenti.

3.2.3. Fun Master Ed

Fun Master Ed munkái számomra külön kiemelkednek az ökoszisztéma-szimulációk közül azért, hogy nem ragaszkodnak a konvencionális, valósághű ábrázolásmódhoz, hanem absztrakt modellek segítségével vizsgálnak komplex biológiai és pszichológiai jelenségeket.

Első munkájában [2] a hangsúly a szociális és érzelmi alapú viselkedésmodellezésen van. A szimuláció olyan változókat vezet be, mint a „boldogság”, valamint a passzív és agresszív viselkedési attitűdök, vizsgálva ezek hatását a populáció stabilitására és a ragadozó-préda interakciókra. Bár az én fejlesztésem elsősorban a túlélési és reprodukciós stratégiákra fókuszál az érzelmi alapú állapotok helyett, ez a megközelítés rávilágított arra, hogy a döntési folyamatokat érdemes belső, rejtett paraméterekkel is befolyásolni, ami némiképp rokonítható a nálam alkalmazott Fuzzy Cognitive Map belső állapotaival.

A szerző későbbi projektje [3] egy másik kritikus irányba, a morfológiai adaptáció felé mutat. Ebben a szimulációban az ágensek testfelépítése (például a lábak száma vagy a nyak magassága) közvetlenül a környezeti változásokra adott válaszként, az evolúció útján alakul ki. Jó példa erre a zsiráfok evolúciója: a magasabban elérhető táplálékforrás szelekciós nyomást gyakorolt a nyakhossz növekedésére.

Bár a jelen szimuláció ragadozó-préda modellje (rókák és nyulak) konkrét biológiai analógiákra épít, a hivatkozott munkák rávilágítottak az absztrakt modellezés előnyeire és a környezethez való fizikai/mentális alkalmazkodás fontosságára. Ezen inspirációk hatására alakítottam ki a döntéshozatali logikát moduláris módon: a rendszeremben az ágensek „gondolkodásmódja” (FSM, Utility vagy FCM) és a hozzájuk tartozó genetikai paraméterek (**Genome**) elválnak a fizikai reprezentációtól. Ez a struktúra biztosítja, hogy a jövőben ne csak előre rögzített fajok, hanem dinamikusan változó viselkedésmintájú és testfelépítésű ágensek is benépesíthessék a szimulációs teret.

3.2.4. Primer

A technikai implementációk mellett Primer youtube csatorna videósorozata [7] nyújtott jelentős elméleti háttérrel a szimuláció biológiai szabályrendszerének kialakításához. Primer munkásságának fő fókusza nem a vizuális komplexitás, hanem az evolúciós folyamatok matematikai és játékelméleti modellezése. Szimulációiban az ágensek (úgynevezett „Blob”-ok) egyszerű szabályok szerint mozognak, azonban a mutációk révén kialakuló tulajdonságok — mint a sebesség vagy a méret — közvetlen hatással vannak az energiafelhasználásra és a túlélési esélyekre.

Az ő megközelítése segített validálni a saját modellmben alkalmazott „trade-off” mechanizmusokat. Primer rávilágított arra, hogy egy sikeres szimulációban minden előnyös

tulajdonságnak (pl. nagyobb sebesség) rendelkeznie kell egy hátránnyal is (pl. magasabb alapanyagcsere-igény), különben a populáció instabillá válik. Bár Primer megoldásai absztrakt, rácsalapú vagy egyszerűsített 2D-s tereken futnak, a tőle átvett elméleti összefüggéseket én egy komplex, 3D-s Unity környezetbe ültettem át. Ez a váltás lehetővé tette, hogy az ő statisztikai alapú következtetéseit egy sokkal dinamikusabb, fizikai alapú interakciókkal (pl. tényleges ütközések, domborzati viszonyok) rendelkező rendszerben vizsgáljam tovább.

3.3. Következtetések és tervezési irányelvek

Az irodalomkutatás, valamint a létező szoftveres megoldások és devlogok elemzése alapján egyértelművé vált, hogy a biológiailag hiteles és szoftvertechnológiailag is fenntartható ökoszisztéma-szimuláció egyedi mérnöki megközelítést igényel. A vizsgált munkákból levont tanulságokat az alábbi konkrét tervezési döntések formájában integrálom a saját rendszerembe:

1. **A döntési mechanizmusok hibridizációja:** Gras és szerzőtársai [5] munkája igazolta a Fuzzy Cognitive Map (FCM) fölényét a merev struktúrákkal szemben, míg Tiago és Reis [4] a diszkrét logikai döntések evolúcióját vizsgálták. E két megközelítést szintetizálva a saját rendszeremben nem egyetlen modellt alkalmazok, hanem egy olyan összehasonlító keretrendszert tervezek, amely az FSM, a folytonos genetikai térrel rendelkező Utility AI és az FCM modellek teljesítményét azonos környezetben, számszerűsíthető módon veti össze.
2. **A polling kiváltása eseményvezérelt architektúrával:** Sebastian Lague [6] megoldásának kritikai elemzése rávilágított arra, hogy a játékmotorok hagyományos `Update()`-alapú polling mechanizmusai komoly teljesítménybeli korlátot jelentenek magas ágensszám mellett. A skálázhatóság érdekében a saját rendszerem tervezésekor alapvető elvárás a teljes körű eseményvezéreltség (C# event-driven megközelítés), ahol az ágensek érzékelési és döntési ciklusai csak indokolt környezeti változások hatására futnak le.
3. **Komplex térbeli és morfológiai modellezés:** Szemben a Bearss [1] által bemutatott hagyományos, kétdimenziós rácsalapú modellekkel, jelen munka a Unity3D motor lehetőségeit kihasználva folytonos 3D-s térben, procedurálisan generált domborzaton valósul meg. VAV [8] és Fun Master Ed [3] munkáiból kiindulva a fizikai környezet (domborzat) és a funkcionális tulajdonságok kapcsolatát úgy tervezem meg, hogy a terepviszonyok közvetlen szelekciós nyomást gyakoroljanak az olyan öröklődő tulajdonságokra, mint a sebesség és az energiafelhasználás.
4. **Biológiai egyensúly és trade-off mechanizmusok:** Primer [7] szimulációi matematikai pontossággal igazolták, hogy a populáció stabilitásához elengedhetetlen az egymással versengő tulajdonságok (trade-offok) bevezetése. A **Genome** osztály tervezésekor ezt az elvet alkalmazom: a nagyobb mozgási sebesség vagy a kiterjedtebb látómező magasabb alapanyagcsere- és energiaköltséggel fog járni, megakadályozva az irreális „szuper-ágensek” kialakulását és a szimuláció korai összeomlását.
5. **Telemetria és tudományos megfigyelhetőség:** Míg a vizsgált YouTube-projektek többsége [6, 8, 2] beért a vizuális reprezentációval vagy egyszerű lokális naplózással, egy MSc szintű kutatás megköveteli az adatok tudományos igényű validálását. Emiatt a rendszer alapvető részévé teszek egy HTTP-alapú külső telemetria csatlakozót, amely az adatokat InfluxDB idősoros adatbázisba rendezi, és

Grafana dashboardokon keresztül teszi mérhetővé a Lotka–Volterra-féle populáció-oszcillációkat.

4. fejezet

A szimulációs rendszer architektúrája és technikai megvalósítása

A specifikációban rögzített funkcionális és nem-funkcionális követelmények elérése érdekében a szimulációs keretrendszer tervezése és implementációja során kiemelt figyelmet fordítottam a modern szoftvertechnológiai minták és az optimális játékmotor-specifikus megoldások alkalmazására.

Az MSc szintű diplomaterv célkitűzéseinek megfelelően a fejlesztés fókuszában a monolitikus struktúrák felbontása, a tiszta komponens-alapú aggregáció, a futásidejű döntéshozatali modellek absztrakciója, valamint a skálázhatóságot garantáló eseményvezérelt működés állt. Ez a fejezet részletesen bemutatja a rendszer szoftverarchitektúráját, az ágensek belső dekompozícióját, a három integrált döntési modellt, valamint az elosztott aszinkron telemetria technikai implementációját.

Ez a fejezet részletesen bemutatja a rendszer szoftverarchitektúráját, az ágensek belső dekompozícióját és az alkalmazott kommunikációs modelleket a megvalósított forráskód alapján.

4.1. Architekturális tervezés és osztályhierarchia

A projekt fejlesztése során a legfőbb szoftvertechnológiai kihívást az ágensek belső felépítésének menedzselése jelentette. A játékmotor-fejlesztésben gyakran előforduló „Istenszerű osztály” (*God Class*) antipattern elkerülése érdekében a rendszer egy hibrid, absztrakcióra és komponens-aggregációra épülő architektúrát kapott, amely szétválasztja az adathordozó, az érzékelő és a végrehajtó rétegeket.

4.1. ábra. Az ágensek komponens-alapú aggregált osztálystruktúrája

A hierarchia csúcsán a Unity `MonoBehaviour`-jából származtatott absztrakt `Animal` osztály áll, amely nem közvetlenül implementálja a biológiai és fizikai funkciókat, hanem összefogja, konfigurálja és koordinálja a specializált részkomponenseket. Az `Animal` ősoosztály felelőssége a közös állapotváltozók, a térbeli referenciák, valamint a genotípushoz kapcsolódó alapvető tulajdonságok tárolása.

A tényleges működési logikát a finomabb granularitású, jól elszigetelt komponensek valósítják meg:

1. **Fiziológiai komponensek** - a belső biológiai szükségletek és az életciklus fenntartásáért felelős modulok gyűjteménye:

- **Age:** az egyed folyamatos öregedését és fiatalkori növekedését vezérli a `lifeSpan` és `adultSize` paraméterek alapján
- **Eat:** a `currentHunger` értéksökkenését menedzseli, kritikus szintnél kiváltja az `OnHungerCritical` eseményt, illetve kezeli a táplálkozással járó érték és státusz változásokat, illetve frissíti a kapcsolódó `HungerBar` vizuális elemet
- **Drink:** az `Eat` komponenshez hasonlóan működik a `currentThirst` változó mentén, vízhiány esetén az `OnThirstCritical` triggerelésével
- **Rest:** a fáradtságot és a pihenési hajlandóságot szabályozza, úgy, hogy tesz egy valószínűségi alapú döntést (minél fáradtabb az egyed, annál valószínűbb, hogy megáll pihenni), amivel pihenés állapotba kényszerítheti az ágenszt

2. **Reprodukciós komponensek** - az evolúciós motor alapját képező, párválasztási és utódnemzési folyamatokat tömörítő réteg:

- **Mate:** kezeli a nemi érettséget, a párzási kényszer szintjét, a cooldown fázisokat, valamint az `IsAcceptable()` metódussal eldönti, hogy a kiszemelt partner vonzereje, az aktuális szükségletek mellett elegendő-e az elfogadáshoz
- **Reproduction:** kizárólag a nőstény egyedeknél aktív modul, sikeres párzás után menedzseli a terhességi időt, illetve kezeli az utódok világrahozatalát a szülők értékei alapján

3. **Interakciós és fizikai komponensek** - az ágens környezetbe ágyazottságát, térbeli helyváltoztatását és észlelését biztosító rendszerek:

- **Movement:** a fizikai helyváltoztatást valósítja meg az ágens állapota alapján, célirányos mozgás (`Move`) vagy menekülés (`Flee`) esetén 5x-ös rotációs sebességszorozót alkalmaz forduláshoz, a céltalan bolyongással szemben (`Wander`)
- **Sensor:** a környezet folyamatos pásztázásáért felelős modul, dinamikusan kalkulálja a ragadozók észlelési esélyét a távolság, valamint a prédá `camouflage` és `stealth` genetikai tulajdonságai alapján
- **Die:** az elhullás végpontja, a fizikai `GameObject`, valamint a hozzátartozó `destructibles` elemek memóriából való takarításáért felelős, illetve `DebugLogger`rel regisztrálja az állat statisztikáit, és elhalálozás körülményeit

A kiterjeszthetőséget és az *Open/Closed* elv érvényesülését az absztrakt metódusok (például `getTargetLayerToMate()`) biztosítják. Ezek révén a specifikus fajok, mint a `Bunny` és a `Fox`, destruktív kódmódosítás nélkül, deklaratív módon határozzák meg a saját rétegmaszk-alapú interakcióikat a Unity `LayerMask` rendszerében:

```
public class Bunny : Animal, IEdible
{
    ...
    public override int getTargetLayerToMate()
    {
        return LayerMask.GetMask("Bunny");
    }
    ...
}
```

4.1. kódrészlet. Nyulak esetén a célréteg kiválasztása

A környezeti elemek absztrakcióját az `IEdible` interfész biztosítja. Ez a tervezési mintázat lehetővé teszi, hogy a ragadozók és a növényevők egységesen kezeljék a táplálékforrásokat:

A `Plant` osztály és a préda szerepet betöltő `Bunny` osztály egyaránt implementálja az `IEdible` interfészt. Így amikor egy ágens táplálkozni szeretne, a `targetRef` entitáson keresztül közvetlenül eléri a metódusait (például a `nutrition` értéket, ami meghatározza, hogy az elfogyasztott egyed mennyivel fogja csökkenteni az állat éhségérzetét), függetlenül attól, hogy növényi vagy állati eredetű biomasszáról van szó:

```
public void Eating()
{
    if (animal.targetRef != null && animal.targetRef.TryGetComponent<IEdible>(out var edibleFood))
    {
        // Egységes tápérték lekérdezés forrástól függetlenül:
        currentHunger = Mathf.Min(currentHunger + edibleFood.getNutrition(), maxHunger);

        // Az célobjektum értesítése az elfogyasztásról:
        edibleFood.Consumed();

        // Táplálkozással kapcsolatos értékek visszaállítása default értékekre:
        animal.targetRef = null;
        animal.sensor.targetMask = LayerMask.GetMask("None");
    }
    else
    {
        // Ha esetleg a célobjektum már nem létezne, mert pl. megette más korábban, mint elértük
        volna, folytatjuk a keresést:
        (animal.status, animal.prevStatus) = (animal.prevStatus, Status.WANDER);
        animal.targetRef = null;
    }
}
```

4.2. kódrészlet. Polimorfikus táplálkozáskezelés az `IEdible` interfészen keresztül

4.2. A ragadozó-préda dinamika kiterjesztése

A szimulációs keretrendszer MSc fázisú továbbfejlesztésének egyik legfontosabb mérföldköve az egyfajos (kizárólag prédát tartalmazó) modellből egy komplexebb, többfajos ökoszisztéma kialakítása volt. Az alaphelyzet kiterjesztése során a rendszer három fő entitásosztályra tagozódik:

- **a primer termelőkre:** (növényzet/táplálék),
- **a primer fogyasztókra:** nyulak, mint préda,
- **a szekunder fogyasztókra:** rókák, mint ragadozók.

A ragadozó faj (róka) bevezetése nem csupán biológiai, hanem komoly szoftverarchitektúráis kihívást is jelentett. Mivel a BSc fázisból származó alaprendszer monolitikus jegyeket hordozott, az új faj integrálását egy átfogó strukturális refaktorálás előzte meg. Az ágensek viselkedésének és belső logikájának elemzése során első lépésként a korábbi általános ágenskódból kiszervezésre kerültek a kifejezetten nyúl-specifikus attribútumok és állapotátmenetek egy dedikált `Bunny` osztályba. Ez a lépés alapozta meg azt a tiszta osztályhierarchiát, amely lehetővé tette, hogy az új `Fox` osztály implementálása minimális kódmódosítással, az alapértelmezett ágens-komponensek újrahasznosításával valósulhasson meg.

A komponensalapú architektúra mellett a skálázhatóságot az interfész-vezérelt tervezés (*Interface-Driven Design*) biztosítja. A táplálkozási lánc modellezésére bevezetésre került az `IEdible` interfész. Ez az absztrakció egységesíti a szimulációban elfogyasztható entitások kezelését.

```
public interface IEatable
{
    float GetNutritionalValue();
    void OnConsumed();
}
```

4.3. kódrészlet. Az IEatable interfész vázlatos struktúrája

Az IEatable interfészt minden osztály implementálja, ami elfogyasztható, legyen szó növényi táplálékról, vagy a nyúl (**Bunny**) ágensekről. Ennek köszönhetően a ragadozó vagy préda ágens táplálékszerző és célpont-választó algoritmusai teljesen függetlenné válhattak a célpont konkrét típusától. Ha a jövőben a rendszer egy újabb trofikus szinttel bővülne (például egy csúcsragadozóval, amely a rókákra vadászik), a **Fox** osztály szintén ellátható az IEatable interfésszel anélkül, hogy a globális interakciós logikát újra kellene írni.

Az új ragadozó entitás megjelenése a préda viselkedési mintáinak bővítését is megkövetelte, amely egy új ágensállapot, a menekülés (**FLEE**) bevezetését jelentette. A hatékonyság érdekében a prédaágensek nem minden frame-ben futtatnak le komplex környezetanalízist, hanem egy diszkrét időzítő alapján, periodikusan (X időközönként) vizsgálják a szenzoros környezetüket. Amennyiben a szenzormaszkk ragadozót detektál, a préda azonnal megszakítja az aktuális (például keresési vagy táplálkozási) folyamatát, és menekülő fázisba lép.

A menekülési vektor meghatározása kritikus fontosságú a túlélés szempontjából. Amennyiben egy nyúl látóterében csupán egyetlen ragadozó tartózkodik, a menekülés iránya a ragadozó pozíciójából a préda pozíciójába mutató vektorral ellentétes. Összetettebb szituációkban, amikor a préda környezetében egyszerre több ragadozó is jelen van, akkor a rendszer egy súlyozott vektorösszegzést alkalmaz az optimális menekülési útvonal kiszámítására.

Ha a nyúl környezetében lévő ragadozók pozícióvektorai $\vec{P}_{fox,i}$ és a nyúl pozíciója $\vec{P}_{bunny}$, akkor az egyes ragadozóktól vett távolodási irányvektorok:

$$\vec{D}_i = \vec{P}_{bunny} - \vec{P}_{fox,i} \quad (4.1)$$

Az eredő menekülési irányvektort ($\vec{V}_{flee}$) ezen vektorok normalizált összege (vagy a távolsággal fordítottan arányos súlyozott összege) adja meg, biztosítva, hogy az ágens geometriailag a legbiztonságosabb, a fenyegetésektől átlagosan legtávolabbi pályát válassza:

$$\vec{V}_{flee} = \text{normalize} \left(\sum_i 1^n \vec{D}_i \right) \quad (4.2)$$

Ez a matematikai megközelítés garantálja, hogy az ágens képes dinamikusan navigálni csapdahelyzetekben is, elkerülve, hogy az egyik ragadozótól menekülve közvetlenül egy másik torkába fusson.

4.3. Eseményvezérelt kommunikációs modell

A szimuláció skálázhatóságát és valós idejű futását a leválasztott, eseményvezérelt kommunikációs architektúra garantálja, amely teljes mértékben felszámolja a képkocka-alapú folyamatos lekérdezést (*polling*). Az ágensek belső és külső eseményeinek szinkronizációjért egy dedikált komponens, az **AnimalEventHandler** felel, amely hídként működik a fiziológiai komponensek és az **Animal** központi állapotgépe között.

4.2. ábra. Eseményvezérelt állapotátmenet szekvenciadiagramja

Az `Animal` osztály a `Subscribe()` metódusban iratkozik fel a részkomponensek által publikált C# eseményekre (`Action`), amelyeket az `AnimalEventHandler` delegált metódusai kezelnek le. Ez a felépítés drasztikusan csökkenti a CPU terhelését, mivel a szimulált egyedek nem futtatnak felesleges ellenőrzéseket a `Unity Update()` ciklusában. Az állapotváltozások diszkrét, eseményalapú triggerelése a következő főbb láncolatokon keresztül zajlik:

- **Fiziológiai krízishelyzetek:** Az `Eat` és `Drink` komponensek a `Step()` metódus periodikus lefutásakor ellenőrzik a belső értékeket. Amint az éhség vagy szomjúság eléri a kritikus szintet, kiváltódik az `OnHungerCritical` vagy `OnThirstCritical` esemény.

Az `AnimalEventHandler` ezen értesítések hatására azonnal átállítja az ágens `sensor.targetMask` értékét a megfelelő célobjektumra (például nyúl esetén `BunnyFood`, róka esetén `Bunny` lesz a réteg neve), és a státuszt `Status.SEARCH` állapotba kényszeríti.

Amennyiben az adott szükséglet teljesen kimerül, a kapcsolódó esemény közvetlenül a `die.HandleDeath()` metódust hívja meg, lezárva az egyed életciklusát.

- **Érzékelési és interakciós triggerek:** Amikor a `Sensor` komponens releváns célpontot azonosít, az `OnTargetSpotted` esemény aktiválódik:

Az `HandleTargetSpotted()` metódus ekkor az aktuális állapotot elmenti a `prevStatus` változóba, és az ágens `Status.MOVE_TOWARDS` fázisba lépteti.

- **Környezeti és fizikai végpontok:** Az olyan fatális környezeti hatások, mint a fulladás (`OnDrowned` esemény), vagy az öregedési limit elérése (`OnAgeLimitReached`) közvetlenül az eseménykezelőn keresztül hajtják végre a megsemmisítést és a statisztikai adatok kiküldését.

Az ágens központi `Decide()` metódusa már erre a tiszta, események által előkészített állapotstruktúrára támaszkodik. A metódus első lépésben, amennyiben vannak regisztrált ragadozók (`predators`), ellenőrzi a veszélyt (`CheckForPredators()`). Amennyiben fenáll a veszély (ragadozó lépett egy meghatározott távolságon belülre), az egyed a `bravery` (bátorság) genetikai tulajdonsága alapján egy valószínűségi döntést hoz, és szükség esetén menekülés (`Status.FLEE`) állapotba vált.

Ha nincs közvetlen veszély, a `Decide()` metóduson keresztül navigálja az ágens adott döntési folyamatnak megfelelően (FSM, Utility vagy FCM).

Ez a megközelítés garantálja, hogy a drága térbeli távolságszámítások csak akkor és ott futnak le, amikor az eseményvezérelt réteg már célirányosan egy objektum felé navigálta az egyedet.

Az `AnimalEventHandler` osztályban központosított eseményvezérelt logikára, valamint a fiziológiai krízishelyzetek és az érzékelési triggerek lekezelésére az alábbi kódrészlet mutat példát:

```
public class AnimalEventHandler : MonoBehaviour
{
    ...

    public void HandleHungerCritical()
    {
        if (animal.status == Status.SEARCH || animal.status == Status.REST)
        {
            // Dinamikus rétegmaszk-váltás a fajspecifikus táplálékra:
            animal.setTargetLayerToEat();
            animal.status = Status.SEARCH;
        }
    }
}
```

```

    }

    public void HandleTargetSpotted()
    {
        if (animal.status == Status.SEARCH || animal.status == Status.WANDER)
        {
            // A korábbi állapot mentése és célirányos mozgásra váltás:
            animal.prevStatus = animal.status;
            animal.status = Status.MOVE_TOWARDS;
        }
    }

    public void HandleHungerDepleted()
    {
        // Közvetlen fatális végpont meghívása polling nélkül
        animal.die.HandleDeath(CauseOfDeath.HUNGER);
    }
}

```

4.4. kódrészlet. Eseménykezelő delegált metódusok

4.4. A döntési folyamatok moduláris tervezése és implementációja

Az ökoszisztéma-szimulációban részt vevő ágensek túlélési esélyeit és viselkedésének realizmusát alapvetően meghatározza a döntéshozatali mechanizmusuk. Annak érdekében, hogy a különböző mesterséges intelligencia architektúrák (FSM, Utility AI, FCM) hatékonysága, számítási igénye és populációdinamikára gyakorolt hatása transzparens módon összehasonlítható legyen, a rendszert moduláris, cserélhető struktúrában kellett megtervezni.

A szoftverarchitektúra gerincét egy közös vezérlő, a `DecisionController` alkotja, amely futásidőben delegálja a döntési feladatokat az enum típusú `DecisionMode`-nak megfelelően. Ez az úgy nevezett *Strategy tervezési minta*, amely lehetővé teszi, hogy egy objektum (esetünkben az `Animal` osztály példányai) viselkedését futásidőben dinamikusan változtassuk meg anélkül, hogy magát az osztályt vagy az azt hívó környezetet módosítani kelljen

4.3. ábra. A döntéshozatali modellek Strategy tervezési mintára épülő struktúrája

Az absztrakció alapját képező központi elágazási logikát és a stratégiák hívását az alábbi implementáció mutatja be:

```

public class DecisionController
{
    public DecisionMode mode;
    public ClassicFSMDecision classic;
    public UtilityDecision utility;
    public FCMDecision fcm;

    public Status Decide(Animal a)
    {
        switch (mode)
        {
            case DecisionMode.ClassicFSM: return classic.Decide(a);
            case DecisionMode.Utility: return utility.Decide(a);
            case DecisionMode.FCM: return fcm.Decide(a);
            default: return Status.WANDER;
        }
    }
}

```

4.4.1. Véges Állapotú Gép (ClassicFSM) megvalósítása

A legegyszerűbb, determinisztikus megközelítést a Véges Állapotú Gép (*Finite State Machine* - *FSM*) képviseli. Ebben az architektúrában az ágens viselkedése diszkrét állapotokra van osztva, és az állapotok közötti átmeneteket mereven rögzített szabályok (if-then struktúrák) vezérlik az ágens belső monitorozott változói (éhség, szomjúság, stamina) és környezeti ingerei (észlelt veszély) alapján.

A szimuláció alapvető állapotait és azok átmeneti feltételeit a 4.1. táblázatban összefoglalt állapot-átmeneti mátrix reprezentálja.

4.1. táblázat. FSM állapot-átmeneti mátrix és kiváltó események

Kiinduló állapot	Célállapot	Kiváltó feltétel / Esemény
WANDER	SEARCH	Belső igény kritikus szint alá esik (éhség, szomjúság).
WANDER / SEARCH	FLEE	Ragadozó észlelése a szenzoros rádiuszon belül.
SEARCH	MOVE_TOWARDS	Megfelelő célpont (táplálék, víz, partner) detektálása.
MOVE_TOWARDS	WANDER	Célpont elfogyasztása / sikeres szaporodás / cél elvesztése.
Bármely állapot	REST (Pihenés)	A stamina (energia) szintje teljesen kimerül ($= 0$).

Bár az FSM rendkívül alacsony számítási kapacitást igényel és könnyen átlátható, merevsége miatt nem képes komplex, árnyalt viselkedési formák vagy egyéni adaptációs stratégiák modellezésére, mivel nem mérlegeli az egymással versengő szükségletek prioritását.

4.4.2. Hasznossági alapú döntési rendszer (Utility AI)

A hasznossági alapú döntési rendszer (*Utility AI*) szakít a bináris döntési szabályokkal, és matematikai megközelítést alkalmaz az ágens optimális cselekvésének kiválasztására. Minden lehetséges állapothoz vagy cselekvéshez egy $[0, 1]$ intervallumba eső *hasznossági értéket* (U) rendel, és az ágens mindig a legmagasabb értékű tevékenységet választja.

A döntés megalapozásához az ágens nyers belső állapotváltozóit először normalizálni kell. Ha v a vizsgált belső érték (például az aktuális éhség), és v_{max} a maximális kapacitás, a normalizált szükséglet (n) kifejezése:

$$n = 1.0 - \left(\frac{v}{v_{max}} \right) \quad (4.3)$$

Ezáltal az n érték minél közelebb van az 1.0-hez, a szükséglet annál sürgetőbb (például a teljesen üres gyomor esetén $n = 1.0$).

A konkrét architektúrában az egyes motivációk végső hasznosságát a normalizált szükségletek és az egyed genetikai állományából származó súlyok (*genes/weights*) lineáris kombinációja határozza meg:

$$U_{action} = w_{action} \cdot n_{action} \quad (4.4)$$

Ahol $w_{action} \in \mathbb{R}^+$ a viselkedési gén súlya (például `hungerWeight`). A döntési lépés során a rendszer kiértékeli az összes releváns cselekvést ($U_{eat}, U_{drink}, U_{rest}, U_{mate}$), majd a kiválasztási függvény az abszolút maximumot keresi meg:

$$\text{Selected_Status} = \arg \max_{action} (U_{action}) \quad (4.5)$$

A matematikai modell C# nyelvű leképezése és a célobjektumok maszkolása a `UtilityDecision` osztályban az alábbi módon valósul meg:

```
public class UtilityDecision
{
    ...

    public Status Decide(Animal a)
    {
        float hungerNormalized = 1f - (a.eat.currentHunger / 100f);
        float thirstNormalized = 1f - (a.drink.currentThirst / 100f);
        float energyNormalized = 1f - (a.rest.currentStamina / a.rest.maxStamina);
        float mateNormalized = 1f - (a.mating.currentMatingUrge / 100f);

        float eatU = genome.hungerWeight * hungerNormalized;
        float drinkU = genome.thirstWeight * thirstNormalized;
        float restU = genome.energyWeight * energyNormalized;
        float mateU = genome.mateWeight * mateNormalized;

        float max = Mathf.Max(eatU, drinkU, restU, mateU);

        if (max == restU) return Status.REST;

        if (max == eatU) a.setTargetLayerToEat();
        else if (max == drinkU) a.sensor.targetMask = LayerMask.GetMask("Drink");
        else a.setTargetLayerToMate();

        return Status.SEARCH;
    }
}
```

4.6. kódrészlet. A hasznossági alapú döntési logika implementációja

Ez a módszer rugalmas, folyamatos átmenetet biztosít a viselkedésformák között, és lehetővé teszi, hogy az egyedek egyéni genetikai preferenciák alapján (pl. egy bátrabb vagy falánkabb egyed más súlyokkal dolgozik) reagáljanak a környezetre.

4.4.3. Fuzzy Kognitív Térkép (FCM) tervezése

A legösszetettebb, nemlineáris döntési modellt a Fuzzy Kognitív Térkép (*Fuzzy Cognitive Map* - *FCM*) adja. Az FCM egy irányított gráf, ahol a csomópontok (C_i) az ágens belső állapotait, környezeti észleléseit vagy közvetlen cselekvési szándékait (outputok) reprezentálják, a köztük lévő élek (W_{ji}) pedig a kauzális kapcsolatok irányát és erősségét határozzák meg a $[-1, 1]$ tartományban.

4.4. ábra. A döntéshozatali Fuzzy Kognitív Térkép hálózati topológiája

Az iterációs lépések során a csomópontok aktivációs szintje folyamatosan frissül a szomszédos csomópontok hatásainak és a korábbi saját értéküknek a figyelembevételével. A matematikai transzformáció leírása az $t + 1$ -edik iterációban:

$$A_i(t + 1) = f \left(A_i(t) + \sum_{j=1}^N A_j(t) \cdot W_{ji} \right) \quad (4.6)$$

Ahol $f(x)$ egy nemlineáris kényszerítő függvény, amely jelen esetben a $[0,1]$ tartományra normalizáló logisztikus szigmoid függvény (`Mathf.Clamp01`), amely megakadályozza az értékek divergálását a hálózaton futó gerjesztési ciklusok során.

A gráf matematikai működését leképező önálló motor és az értékek frissítése az alábbi kód szerint fut le:

```
public class FCModel
{
    private Dictionary<string, FCNode> nodes = new Dictionary<string, FCNode>();
    private List<FCMLink> links = new List<FCMLink>();

    public void Update()
    {
        foreach (var node in nodes.Values)
            node.nextValue = 0f;

        // Kauzális élek mentén törtéőn értékgerjesztés:
        foreach (var link in links)
            link.to.nextValue += link.from.value * link.weight;

        // Értékek szorítása és matematikai normalizálása:
        foreach (var node in nodes.Values)
            node.value = Mathf.Clamp01(node.nextValue);
    }
}
```

4.7. kódrészlet. FCM hálózati frissítési ciklus (FCM.cs)

Az `FCMDecision` osztály integrálja ezt a hálózatot a Unity fizikai környezetébe, frameenként táplálva a bemeneteket és kiértékelve a domináns kimenetet:

```
public class FCMDecision
{
    private FCModel model;

    public FCMDecision(FCMGenome genome)
    {
        model = FCMFactory.Create(genome);
    }

    public Status Decide(Animal a)
    {
        float hungerNormalized = 1f - (a.eat.currentHunger / 100f);
        float thirstNormalized = 1f - (a.drink.currentThirst / 100f);
        float energyNormalized = 1f - (a.rest.currentStamina / a.rest.maxStamina);
        float fearNormalized = a.spottedThreats.Any() ? 1f : 0f;
        float mateNormalized = 1f - (a.mating.currentMatingUrge / 100f);

        model.SetInput("Hunger", hungerNormalized);
        model.SetInput("Thirst", thirstNormalized);
        model.SetInput("Energy", energyNormalized);
        model.SetInput("Fear", fearNormalized);
        model.SetInput("Mate", mateNormalized);

        model.Update();

        float eat = model.GetOutput("Eat");
        float drink = model.GetOutput("Drink");
        float rest = model.GetOutput("Rest");
        float flee = model.GetOutput("Flee");
        float mate = model.GetOutput("MateAct");

        float max = Mathf.Max(eat, drink, rest, flee, mate);

        if (max == flee) return Status.FLEE;
        if (max == rest) return Status.REST;

        if (max == eat) a.setTargetLayerToEat();
        else if (max == drink) a.sensor.targetMask = LayerMask.GetMask("Drink");
        else a.setTargetLayerToMate();
    }
}
```

```

        return Status.SEARCH;
    }
}

```

4.8. kódrészlet. Az FCM döntéshozatali logika integrációja

4.5. A populációgenetikai adatmodell

Az evolúciós szimuláció sarokköve a populációgenetikai adatmodell, amely az egyedek viselkedési és fizikai jellemzőit digitális kromoszómákká (genomokká) kódolja. A rendszerben a genetikai állomány nem csupán statikus tulajdonságokat határoz meg, hanem közvetlen bemenetként szolgál a döntéshozatali mechanizmusok számára, lehetőséget biztosítva a környezethez való dinamikus adaptációra és a természetes szelekció modellezésére.

4.5.1. A Genome osztályok belső felépítése és kódolása

A moduláris tervezési elveknek megfelelően a genetikai reprezentáció két fő komponensre válik szét a döntési architektúrákhoz igazodva: a `UtilityGenome` és az `FCMGenome` osztályokra.

Megjegyzés a klasszikus FSM architektúrával kapcsolatban: Fontos kiemelni, hogy a hagyományos végesállapotú gép (`ClassicFSM`) döntéshozatali mód nem rendelkezik dedikált genetikai adatmodellel. Mivel a klasszikus FSM állapotátmenetei merev, determinisztikus szabályrendszereken alapulnak, a viselkedési logika nem tartalmaz szabadon konfigurálható súlyparamétereket. Ebből adódóan az FSM-alapú egyedek fenotípusa homogén, nem mutatnak egyedi variabilitást, és nem vesznek részt az evolúciós (mutációs és keresztezési) ciklusban.

A fizikai és döntési paraméterek tárolása (`UtilityGenome`): A `UtilityGenome` osztály az egyedek alapvető belső késztetéseinek (motivációinak) súlyozásáért felelős. Ezen paraméterek határozzák meg, hogy az egyed milyen prioritással reagál a létfontosságú környezeti és fiziológiai változásokra. Az osztály belső struktúrája a következő lebegőpontos (`float`) értékeket tárolja:

- `hungerWeight`: az éhségérzet prioritása a hasznossági függvényben
- `thirstWeight`: a szomjúságérzet súlya a vízkeresési motivációban
- `energyWeight`: a staminaveszteség és a pihenési hajlandóság együtthatója
- `mateWeight`: a szaporodási ösztön dominanciája kedvező körülmények esetén

Ezek a `float` típusú gének homogén, normalizált teret alkotnak, amely közvetlenül skálazza a döntéshozatalkor számított hasznossági értékeket.

```

public class UtilityGenome
{
    public float hungerWeight = 1f;
    public float thirstWeight = 1f;
    public float energyWeight = 1f;
    public float mateWeight = 1f;

    public UtilityGenome Clone()
    {
        return new UtilityGenome
        {
            hungerWeight = this.hungerWeight,
            thirstWeight = this.thirstWeight,

```

```

        energyWeight = this.energyWeight,
        mateWeight = this.mateWeight
    };
}
}

```

4.9. kódrészlet. A UtilityGenome osztály belső felépítése

A kognitív térkép élsúlyainak kódolása (FCMGenome): A Fuzzy Cognitive Map alapú döntéshozatalhoz egy jóval komplexebb, hálózatos genetikai struktúrára van szükség. Az FCMGenome nem nevesített változóként, hanem egy egydimenziós `float[]` tömbként (`weights`) tárolja a gráf élsúlyait.

- A tömb hossza megegyezik a hálózatban definiált kauzális kapcsolatok (élek) számával (jelenlegi konfigurációban 8 aktív él).
- Az élsúlyok értéktartománya a $[-1.0, 1.0]$ intervallumra van korlátozva, ahol a pozitív értékek egyenes, a negatív értékek pedig fordított arányú kauzális kapcsolatot (gátlást) reprezentálnak a koncepciócsomópontok között (például a *Félelem* és az *Évés* közötti kapcsolat gátló hatása).

Az egyedek replikációjakor és inicializálásakor kritikus fontosságú a genetikai állomány mély másolása (deep copy), amelyet mindkét osztályban a `Clone()` tagfüggvény valósít meg, megakadályozva a memóriacímek referenciáinak nemkívánatos átfedését a szülő és az utód egyedek között.

```

public class FCMGenome
{
    public float[] weights;

    public FCMGenome(int linkCount)
    {
        weights = new float[linkCount];
        for (int i = 0; i < linkCount; i++)
            weights[i] = UnityEngine.Random.Range(-1f, 1f);
    }

    public FCMGenome Clone()
    {
        FCMGenome g = new FCMGenome(weights.Length);
        Array.Copy(weights, g.weights, weights.Length);
        return g;
    }
}

```

4.10. kódrészlet. Az FCMGenome osztály felépítése

4.5.2. A mutációs algoritmus tervezése és matematikai modellje

A genetikai variabilitás fenntartásáért és az új viselkedési minták felfedezéséért (exploration) a `Mutation` statikus osztály felel. Az algoritmus két fő hiperparaméter mentén dolgozik: a mutációs ráta (`rate`), amely az esélyt adja meg arra, hogy egy adott gén megváltozik-e, valamint a mutációs erő (`strength`), amely a változás maximális léptékét korlátozza.

A mutáció matematikai modellje egy Gauss-eloszláshoz közelítő uniform perturbáción alapul, amely a meglévő génértéket kis lépésekkel tolja el, megőrizve a korábbi evolúciós eredmények stabilitását (graduális fejlődés). A perturbációs függvény formálisan:

$$g'_i = \begin{cases} g_i + \Delta g, & \text{ha } r < \text{rate} \\ g_i, & \text{egyébként} \end{cases} \quad (4.7)$$

Ahol $r \sim \mathcal{U}(0, 1)$ egy egyenletes eloszlású véletlen szám, a perturbáció mértéke pedig $\Delta g \sim \mathcal{U}(-\text{strength}, \text{strength})$.

A gyakorlati implementáció során a rendszer minden egyes génre külön elvégzi ezt a valószínűségi ellenőrzést. Amennyiben a mutáció sikeresen lezajlik az **FCMGenome** tömbjén, az új súlyok a döntéshozatali ciklusban azonnal módosítják az aktivációs függvények eredőjét, finomhangolva az állat reakcióidejét és túlélési stratégiáját.

4.5.3. Keresztezési (rekombinációs) mechanizmus

A populációgenetikai modell teljességéhez a **Breeding** osztály biztosítja a szexuális reprodukció modellezését. Amikor két egyed sikeresen párosodik, az utód genetikai állománya a szülői gének számtani közepéből (*bblend crossover*) áll elő:

$$g_{child} = \frac{g_{male} + g_{female}}{2} \quad (4.8)$$

A számtani közép alapú rekombináció biztosítja, hogy az utódok a szülők által már sikeresen tesztelt túlélési stratégiák interpolációs terében induljanak el. A rekombinációt közvetlenül követi a fent részletezett mutációs fázis, biztosítva, hogy az utód ne csupán a szülők pontos átlaga legyen, hanem egyedi, potenciálisan jobb adaptációs mutatókkal rendelkező új egyed.

4.6. Elosztott Telemetry és Aszinkron Adatgyűjtés

A szimuláció evolúciós folyamatainak pontos elemzéséhez elengedhetetlen a futás közbeni adatok valós idejű rögzítése. Ugyanakkor egy több száz egyedet mozgató ökoszisztémában a mérési pontok (születések, halálozások, különböző populációs számok...) elmentése jelentős I/O-terhelést jelenthet. Ezért a rendszer az adatokat egy külső idősor-adatbázisba (InfluxDB) továbbítja.

4.6.1. A Unity főszál (Main Thread) védelme

A Unity architektúrájából adódóan a fizikai kalkulációk, a renderelés és a szkriptek alapértelmezetten egyetlen főszálon (*Main Thread*) futnak. Ha a HTTP-kéréseket szinkron módon, a főszálat blokkolva küldenénk el, az az adatbázis válaszidejétől függően komoly fagyásokat (*micro-stuttering*) okozna, ami felborítaná a szimuláció valós idejű dinamikáját.

A probléma elkerülése érdekében az **InfluxLogger** osztály aszinkron módon, Unity *Coroutine*-ok (kooperatív többfeladatosság) segítségével valósítja meg az adatgyűjtést, háttérbe szorítva a hálózati I/O műveleteket. Bár a *Coroutine*-ok technikailag a főszálon ütemeződnek, a végrehajtást darabokra bontják, és a hálózati művelet válaszára való várakozást átadják a háttérben futó hálózati worker szálaknak.

Az adatok naplózását az **InfluxLogger.Log(string line)** statikus metódus indítja meg, amely egy *MonoBehaviour* példányon keresztül ütemezi be a **Send** nevű aszinkron folyamatot:

```
public static void Log(string line)
{
    if (coroutineRunner == null)
    {
        Debug.LogError("InfluxLogger: coroutineRunner missing!");
        return;
    }
    coroutineRunner.StartCoroutine(Send(line));
}

private static IEnumerator Send(string body)
{

```

```

var request = new UnityWebRequest(WriteUrl, "POST");
byte[] bodyRaw = Encoding.UTF8.GetBytes(body);

request.uploadHandler = new UploadHandlerRaw(bodyRaw);
request.downloadHandler = new DownloadHandlerBuffer();

request.SetRequestHeader("Authorization", "Token " + token);
request.SetRequestHeader("Content-Type", "text/plain");

yield return request.SendWebRequest();

if (request.result != UnityWebRequest.Result.Success)
{
    Debug.LogError($"InfluxDB Send Error: {request.error}");
}
}

```

4.11. kódrészlet. Aszinkron adatküldés megvalósítása Coroutine segítségével

A `yield return request.SendWebRequest();` utasítás pontján a Coroutine átadja a vezérlést, így a Unity főszál zavartalanul folytathatja a szimulációs keretek (Frames) számítását és renderelését. Amint a háttérzálon a HTTP POST kérés sikeresen lezajlik, a Coroutine a következő képkockában ott folytatódik, ahol abbamaradt, és ellenőrzi a válasz státuszát.

4.6.2. A HTTP POST kérések felépítése és az alkalmazott InfluxDB Line Protocol

A 2.6.3. fejezetben bemutatott elméleti alapoásra építve, a rendszer tervezési fázisában meg kellett határozni, hogy a szimuláció ágenseinek tulajdonságai miként feleltethetők meg az InfluxDB adatmodelljének. A telemetriai csomagok felépítésének optimalizálása kulcsfontosságú, hiszen a nagyszámú ágens folyamatos adatszolgáltatása jelentős hálózati terhelést jelenthet.

A szimulációs entitások és események leképezése során a következő specifikus mérnöki döntések születtek:

- **Measurementek elkülönítése:** A rendszerben különálló méréseket definiáltam a különböző eseménytípusokhoz. Például a populáció pillanatnyi állapotát a **population**, az egyedek elhalálozását a **death**, míg a táplálékforrásokat a **food** mérés reprezentálja.
- **A Tag-ek megválasztása a nagy kardinalitás elkerülésével:** Mivel az InfluxDB a Tag-eket indexeli, a rosszul megválasztott, túl sűrűn változó értékek (*High Cardinality*) az adatbázis index-robbanásához és teljesítménycsökkenéséhez vezetnének. Emiatt *Tag*-ként kizárólag diszkrét, jól csoportosítható metaadatokat használ a rendszer:
 - **species:** az ágens faja (pl. BUNNY), ami alapján a Grafana dashboardokon azonnali szűrések végezhetők fajonként
 - **cause:** az elhalálozás oka (pl. HUNGER, AGE), ami a mortalitási statisztikák alapja
 - **run_id:** a szimulációs futás egyedi időbélyeg-alapú azonosítója. Ez teszi lehetővé, hogy a Grafana felületén több különböző paraméterezésű kísérlet (futtatás) eredményeit egymással összehasonlítsuk, vagy korábbi futtatások adatait teljesen elszeparáljuk
- **A Field-ek meghatározása az evolúciós metrikákhoz:** Minden olyan attribútum, amely a mutációs algoritmusok hatására folyamatosan változik, vagy matematikai aggregáció (pl. átlagolás) tárgyát képezi, *Field*-ként lett deklarálva.

Ilyenek az egyedek genetikai és aktuális fizikai jellemzői: a sebesség (**speed**), a látótávolság (**sight**), az adott állat vonzereje (**charm**). Ezekben a mezőkön a Grafana futásidőben képes kiszámítani a populációra vonatkozó átlagokat, így kirajzolva a természetes szelekció trendjeit.

- **Időbélyeg (Timestamp) és precízió:** Annak érdekében, hogy a hálózaton továbbított bájtok számát csökkentjük, az alapértelmezett nanoszekundumos precízió helyett másodperces (**precision=s**) felbontást alkalmazunk a kérések küldésekor használt URL-ben, így az időbélyeg rövidebb, UNIX formátumban utazik.

Egy elpusztult nyúl (**Bunny**) adatainak felküldésekor generált Line Protocol sztring felépítése:

```
death,
run_id=20260527_224700,species=BUNNY,cause=CONSUMED age=1.2,speed=5.4,sight=15.0,
lifeSpan=60.0,charm=42.0,pregnancyDuration=10.0,starving=30.0,drying=40.0,triedForBaby=2,
gaveBirth=0,kids=0,step=1420
```

4.12. kódrészlet. Példa InfluxDB Line Protocol formátumú üzenetre

Ez az elrendezés biztosítja, hogy a Grafana dashboardok másodperces frissítés mellett is rendkívül gyorsan képesek legyenek lekérdezni a több ezer adatpontot, hiszen a szűrést végző oszlopok (**species**, **cause**) mind indexelve, míg a grafikonokon ábrázolt trendek (**speed**, **sight**) a rugalmasan aggregálható mezőkből származnak.

5. fejezet

Az eredmények értékelése

5.1. Rendszer- és teljesítményelemzés (Kritikai reflexió)

Az eseményvezérelt architektúra hatása

Az InfluxDB HTTP kapcsolat overheadje?

5.2. A döntési modellek kísérleti összehasonlítása

Melyik gondolkodásmód bizonyult a legstabilabbnak?

A genetikai drift elemzése: hogyan változtak a populáció átlagértékei X generáció után

5.3. Továbbfejlesztési lehetőségek

A kifejlesztett ökoszisztéma-szimuláció egy stabil, moduláris alapkeretet biztosít, amely mind szoftverarchitekturális, mind biológiai modellezési szempontból jelentős továbbfejlesztési potenciállal rendelkezik. A jövőbeli kutatási és fejlesztési irányok három fő kategóriába csoportosíthatók.

5.3.1. Technológiai irány: Skálázhatóság és a Unity DOTS architektúra

A jelenlegi implementáció a hagyományos objektumorientált (*Object-Oriented Programming, OOP*) megközelítést alkalmazza, ahol minden ágens egy egyedi `MonoBehaviour` alapú `GameObject` a Unity szintéren. Bár ez a struktúra ideális a komplex állapotgépek és a Fuzzy Cognitive Map egyedi kiértékelésére, komoly teljesítménybeli korlátokat (*performance bottleneck*) jelent a CPU és a memória-hozzáférés szintjén. A szimulált ágensek száma a hardverkapacitástól függően néhány száz, maximum egy-kétezer egyedben maximalizálódik, mivel az objektumok kezelése és a Unity belső alrendszereinek (pl. `Update()` ciklusok overheadje) jelentős erőforrást igényel.

A szimuláció nagyságrendi skálázhatóságának biztosításához a technológiai jövőkép a **Unity DOTS (Data-Oriented Technology Stack)** és az **ECS (Entity Component System)** architektúra bevezetése.

- **Adatközpontú tervezés (ECS):** A `GameObject`-ek helyett az egyedeket egyszerű azonosítók (*Entities*), a tulajdonságaikat (pozíció, éhség, genetikai súlyok) tiszta adatstruktúrák (*Components*), a logikát pedig globális rendszerek (*Systems*) reprezentálnák. Ez megszüntetné a memóriatöredezettséget, és lehetővé tenné a gyorsítótár-optimalizált (*cache-friendly*) szekvenciális adatolvasást.

- **Párhuzamosítás (C# Job System & Burst Compiler):** Az ECS-re való átállással az egyedek döntéshozatali ciklusai (legyen az hasznossági alapú vagy FCM hálózat) natív gépi kódra fordulnának, és automatikusan szétszthatók lennének a CPU összes elérhető magja között.

Ezen technológiai paradigmaváltás eredményeképpen a szimulációs kapacitás a jelenlegi néhány száz ágensről **több tízezer párhuzamosan működő ágensre** növekedne, ami már valódi, makroszintű populációdinamikai és makroevolúciós trendek vizsgálatát is lehetővé tenné.

5.3.2. Biológiai és ökológiai irány: A szimulációs hűség növelése

A környezeti és viselkedési modell finomhangolásával a szimuláció közelebb vihető a valós ökoszisztémák komplexitásához. A legfontosabb biológiai fejlesztési irányok a következők:

Dinamikus környezeti tényezők:

- **Nappal-éjszaka ciklusok:** Az ágensek látási sugarának (`sensor.radius`) és camouflaze mutatóinak időfüggő skálázása. Ez lehetőséget adna az éjszakai (nokturnális) és nappali (diurnális) életmód evolúciós kialakulására.
- **Szezonális (Évszakok):** A növényzet reprodukciós rátájának és a vízforrások elérhetőségének globális, ciklikus változtatása, amely rákényszerítené a populációt a vándorlásra, az erőforrások felhalmozására vagy a reprodukciós időszakok optimalizálására.
- **Domborzatfüggő mikroklímák:** Az *Island* terep magassági és koordináta-adatai alapján különböző szárazsági vagy hőmérsékleti zónák kialakítása, amelyek befolyásolnák a staminaveszteséget vagy a metabolikus rátát.

Komplex társas és szaporodási interakciók:

- **Falka-viselkedés (Flocking/Swarming):** Csoportos vadászati vagy védekezési stratégiák kódolása. A ragadozók (rókák) képesek lennének koordináltan bekeríteni a zsákmányt, míg a prédák (nyulak) csoportba tömörülve csökkenthetnék az egyéni predációs kockázatot.
- **Utódgondozás és fészekrakás:** A szaporodási mechanizmus kiterjesztése fix vagy dinamikus épített objektumokkal (pl. nyúlüregek, kotorékok). A vemhes nőtények nem a nyílt szintéren hoznák világra az utódokat, hanem védett környezetben, ahol a kölykök egy bizonyos fejlődési fázisig (`currentAge < 1`) védve lennének a predációtól, miközben a szülők táplálékot hordanak számukra.

Ezen irányok bevezetése nemcsak a szimuláció vizuális és funkcionális komplexitását növelné, hanem új dimenziókat nyitna az adaptív ágensok genetikai konvergenciájának és a mesterséges élet (*Artificial Life*) kutatásának területén.

5.3.3. Játékmenet és gamifikáció: Interaktív ökoszisztéma-szimulátor és Sandbox mód

Mivel a keretrendszer natívan a Unity játékmotorra épül, a szimuláció funkcionális kiterjesztésének egy harmadik, magától értetődő iránya a projekt gamifikációja (*gamification*). Ez a megközelítés a tiszta tudományos modellezést egy interaktív, oktatási vagy szórakoztató célú „God game” vagy túlélőjáték-mechanizmussá alakítaná át.

Interaktív Sandbox és paraméter-tuning: A jövőbeli fejlesztések során megvalósítható egy olyan felhasználói felület (UI), amelyen keresztül a felhasználó valós időben, közvetlenül beavatkozhat az ökoszisztéma működésébe. A játékos maga konfigurálhatná a kezdeti génkészleteket, manuálisan helyezhetne el új egyedeket, vagy akár globális környezeti katasztrófákat (pl. aszály, járványok) idézhetne elő, tesztelve a populációk túlélőképességét és evolúciós stabilitását.

Közvetlen ágens-irányítás és túlélési mód: A gamifikáció legmagasabb szintje egy olyan hibrid játékmód bevezetése lenne, ahol a felhasználó átvehetné az irányítást egy kiválasztott ágens (például egy nyúl vagy egy róka) felett. Ebben a túlélési módban a játékosnak magának kellene menedzselnie az egyed belső szükségleteit (éhség, szomjúság, staminavesztés), miközben a környezetében lévő többi ágens továbbra is autonóm módon működik.

Döntéshozatali architektúrák aszimmetrikus versenye: Ez a megközelítés egy egyedülálló, aszimmetrikus versenyt eredményezne a humán intelligencia és a beépített mesterséges intelligencia-modellek között. A játékos közvetlenül összemérhetné túlélési stratégiáját és reflexeit:

- a merev, szabályalapú, de kiszámítható **ClassicFSM** ágensekkel,
- a pillanatnyi prioritásokat matematikailag súlyozó **Utility**-alapú egyedekkel, valamint
- a komplex, asszociatív visszacsatolásokat és félelmet is modellezni képes, evolválódott **FCM** hálózatokkal.

Ez a fejlesztési irány nemcsak a szoftver demonstrációs értékét növelné, hanem kiváló alapként szolgálhatna oktatási szoftverekhez (pl. az ökológiai egyensúly és a természetes szelekció szemléltetésére), vagy akár egy komplex, önálló stratégiai játék prototípusaként is megállná a helyét.

5.4. Összefoglalás

Jelen diplomamunka egy négy féléven átívelő, szisztematikus kutatási és szoftverfejlesztési folyamat szintézise, melynek elsődleges célja egy olyan robusztus és moduláris ökoszisztéma-szimulációs keretrendszer létrehozása volt, amely alkalmas populációgenetikai, evolúciós és ágensalapú döntéshozatali modellek tudományos igényű vizsgálatára, valamint valós idejű telemetriai elemzésére.

A fejlesztési folyamat kiindulópontját képező BSc szakdolgozati fázisban egy korlátozott funkcionalitású, monolitikus struktúrájú alkalmazás állt rendelkezésre. Ez a korai rendszer kizárólag egyetlen préda faj autonóm működését modellezte kódgenerált környezetben, merev, determinisztikus állapotátmenetekre támaszkodva, lokális és strukturálatlan adatmentés mellett.

A mesterképzés (MSc) során elvégzett mérnöki munka alapvetően változtatta meg a szoftver irányvonalát: a korábbi korlátokat felszámolva a hangsúly a tiszta szoftver-architektúráis mintákra, a skálázhatóságra, a biológiai hűség növelésére és az elosztott adatgyűjtésre helyeződött át.

A szoftvertechnológiai modernizáció első lépéseként a „god class” antipatternnek felszámolásával egy tiszta, komponens-aggregációra épülő osztályhierarchiát alakítottam ki a Unity3D motor környezetében. A számítási kapacitás optimalizálása és az ágensek közötti decoupling érdekében a rendszert eseményvezérelt kommunikációs modellre ültettem

át. Ez a megközelítés minimalizálta a felesleges, frame-enként lefutó állapotellenőrzéseket, biztosítva a CPU-erőforrások hatékony elosztását növekvő egyedszám mellett is.

A biológiai hűség fokozása érdekében a szimulációs teret procedurálisan generált, folytonos 3D-s domborzattá terjesztettem ki, amely fizikai akadályaival közvetlen szelekciós nyomást gyakorol az ágensekre. Bevezetésre került a ragadozó faj (rókák), ezáltal sikerült digitális környezetben reprodukálni a klasszikus Lotka–Volterra-féle ragadozó-préda populáció-oszcillációkat. Az egyedek viselkedését egy komplex, mutációra és rekombinációra képes genetikai adatmodell (**Genome**) vezérli. A szuper-ágensek kialakulását versengő trade-off mechanizmusokkal akadályoztam meg, ahol a magasabb sebesség vagy látótávolság arányosan növeli az egyed metabolikus energiaköltségét.

A dolgozat egyik legfontosabb tudományos és mérnöki eredménye a döntéshozatali folyamatok moduláris absztrakciója, amely lehetővé tette három radikálisan eltérő döntési paradigma párhuzamos integrációját és összehasonlítását:

- **Véges állapotgép (ClassicFSM):** A szabályalapú, stabil, de futásidőben merev és evolúciós szempontból alkalmatlan megközelítés reprezentációja.
- **Hasznossági alapú döntési rendszer (Utility AI):** Ahol az ágensek a normalizált belső szükségletek és külső ingerek lineáris kombinációjaként számítják ki az akciók prioritását. A súlyokat a **UtilityGenome** hordozza, folytonos adaptációs teret biztosítva a természetes szelekciónak.
- **Fuzzy Kognitív Térkép (FCM):** A legkomplexebb integrált modell, amely egy irányított, súlyozott gráf segítségével, iteratív és nem-lineáris szigmoid aktivációs függvényeken keresztül modellezi a belső állapotok (pl. éhség, félelem) és akciók egymásra hatását a **FCMGenome** súlymátrixa alapján.

A szimulációk során keletkező adatok tudományos kiértékeléséhez egy professzionális ipari telemetriai pipeline-t terveztem és valósítottam meg. A Unity főszálának védelme érdekében aszinkron architektúrát alkalmazva, HTTP POST kéréseken keresztül transzportálok a strukturált populációdinamikai és genetikai adatokat egy külső InfluxDB idősoradatbázisba. Az adatok vizuális reprezentációját, valós idejű statisztikai elemzését és a különböző döntési modellek stabilitásának összehasonlítását egy egyedileg konfigurált Grafana dashboard rendszer biztosítja.

Összességében a kitűzött feladatokat hiánytalanul teljesítettem. A létrehozott szoftverrendszer nem csupán igazolta a biológiai alapelméletek digitális reprezentálhatóságát, hanem egy olyan skálázható és kiterjeszthető mérnöki keretrendszert biztosít, amely szilárd alapot nyújt a jövőbeli kutatásokhoz, legyen szó akár a Unity DOTS alapú masszív párhuzamosítással történő skálázásról, vagy a Sandbox módú gamifikációs továbbfejlesztésekről.

Irodalomjegyzék

- [1] E. Michael Bearss – Walter Alan Cantrell – C. W. Hall – Mikel D. Joy E. Pinckard: Wolf sheep predation: reimplementing a predator-prey ecosystem model as an instructional exercise in agent-based modeling. In *Proceedings of the 2022 ACM Southeast Conference* (konferenciaanyag). 2022, Association for Computing Machinery, 38–43. p. URL <https://dl.acm.org/doi/abs/10.1145/3476883.3520218>.
- [2] Fun Master Ed: Simulating an ecosystem. <https://www.youtube.com/watch?v=I5ICps2a9vo>, 2020. Online videó; legutoljára megtekintve: 2026. május 12.
- [3] Fun Master Ed: Creating an ecosystem simulation game in 6 months. <https://www.youtube.com/watch?v=52ZeHGGEf6o>, 2022. Online videó; legutoljára megtekintve: 2026. május 12.
- [4] Tiago Francisco – Gustavo Miguel Jorge dos Reis: Evolving predator and prey behaviours with co-evolution using genetic programming and decision trees. In *Proceedings of the 10th annual conference on Genetic and evolutionary computation*, GECCO '08 konferenciasorozat. 2008, Association for Computing Machinery, 1893–1900. p. URL <https://dl.acm.org/doi/10.1145/1388969.1388996>.
- [5] Robin Gras – Didier Devaurs – Adrianna Wozniak – Adam Aspinall: An individual-based evolving predator-prey ecosystem simulation using a fuzzy cognitive map as the behavior model. *Artificial Life*, 15. évf. (2009) 4. sz., 423–463. p. URL <https://dl.acm.org/doi/abs/10.1162/artl.2009.Gras.012>.
- [6] Sebastian Lague: Coding adventure: Simulating ecosystems. https://www.youtube.com/watch?v=r_It_X7v-1E, 2019. Online videó; legutoljára megtekintve: 2026. május 12.
- [7] Primer: Evolution. <https://www.youtube.com/playlist?list=PLKortajF2dPBWMIS6KF4RLtQiG6KQrTdB>, 2018–2025. Online videósorozat; legutoljára megtekintve: 2026. május 12.
- [8] VAV: I made an ecosystem simulation in unity. they evolved. (devlog). <https://www.youtube.com/watch?v=Bfcg4tS8hpw>, 2023. Online videó; legutoljára megtekintve: 2026. május 12.